



Universidad de Oviedo

FACULTAD DE CIENCIAS

GRADO EN MATEMÁTICAS

TRABAJO DE FIN DE GRADO

Modelo híbrido de valoración inmobiliaria

Integración de datos estructurados y procesamiento de imágenes mediante
CLIP

Autor:

Anxo Herrera Castro

Tutor:

Enol Junquera Álvarez

Oviedo, 28 de mayo de 2026

Índice general

Índice de Figuras	3
Índice de Siglas	4
1. Introducción y Objetivos	5
2. Fundamentos del Aprendizaje Automático	7
2.1. Inteligencia artificial	7
2.1.1. Aprendizaje supervisado	7
2.1.2. Aprendizaje no supervisado	8
2.1.3. Aprendizaje por refuerzo	8
2.2. Conceptos fundamentales del aprendizaje automático	9
2.2.1. Generalización, sobreajuste y subajuste	9
2.2.2. Estimadores, sesgo y varianza	10
2.2.3. Hiperparámetros y validación	14
2.2.4. Estimación de máxima verosimilitud	14
2.2.5. Desafíos	15
3. Redes Neuronales	17
3.1. La unidad	17
3.2. El perceptrón multicapa	19
3.3. Entrenamiento de una red neuronal	20
3.3.1. Funciones coste	20
3.3.2. Algoritmos de optimización	21
3.3.3. Retropropagación	22
3.4. Teorema de aproximación universal	24
3.5. Regularización	25
3.5.1. Penalización a la norma de los parámetros	26
3.5.2. <i>Data augmentation</i>	27
3.5.3. Parada temprana	28
3.5.4. Dropout	29
3.6. Transformadores	30
3.6.1. Mecanismo de la atención	31
3.6.2. Bloques del transformador	34
3.6.3. La capa de salida	36
3.6.4. Variantes de la arquitectura	37
4. Visión Artificial	38
4.1. ImageNet y el problema del reconocimiento visual	38
4.2. Redes neuronales convolucionales	39

4.3. Transformadores para imágenes	39
4.4. El modelo CLIP	41
4.4.1. Entrenamiento del modelo	42
4.4.2. Uso de CLIP para la predicción <i>zero-shot</i>	43
4.4.3. Ingeniería de <i>prompts</i> y ensamblaje	45
5. Metodología y Diseño Experimental	47
5.1. Exploración y preprocesamiento	47
5.1.1. Obtención de la base de datos	47
5.1.2. Exploración y preprocesamiento de los datos	48
5.1.3. Distribución de la variable precio	48
5.1.4. Detección de valores atípicos	48
5.1.5. Relación entre precio y superficie	50
5.1.6. Análisis de la ubicación	51
5.1.7. Análisis de características estructurales y equipamiento	52
5.1.8. Normalización de variables	53
5.1.9. Codificación de variables categóricas	54
5.2. Metodología propuesta	55
5.2.1. Integración de características	55
5.2.2. Formulación del modelo	56
5.2.3. Función objetivo	57
5.3. Configuración experimental	58
5.3.1. División del dataset	58
5.3.2. Métricas de evaluación	59
6. Resultados y Discusión	61
6.1. Rendimiento predictivo	61
6.2. Contrastes de hipótesis	61
6.3. Diagnóstico y limitaciones del modelo M3	63
6.3.1. Análisis de residuos	63
6.3.2. Interpretabilidad de coeficientes y varianza explicada	63
6.3.3. Análisis de inmuebles singulares	65
6.4. Construcción de un nuevo modelo	66
6.4.1. Regresor de <i>gradient boosting</i>	67
7. Conclusiones y líneas de Trabajo Futuro	69
7.1. Conclusiones	69
7.2. Líneas de trabajo futuro	70
A. Traducción de los prompts utilizados para la evaluación visual mediante CLIP	71
B. Ingeniería de Características	73
C. Hiperparámetros y Reproducibilidad	74

Índice de figuras

1.1. Mapa de Gijón por zonas	6
2.1. Paradigmas de aprendizaje automático	9
2.2. Sobreajuste, subajuste y buen ajuste	10
2.3. Compromiso sesgo-varianza	13
3.1. Representación de una neurona	18
3.2. Funciones de activación	18
3.3. Estructura de un perceptrón multicapa	20
3.4. Comparación entre mínimos locales y globales	22
3.5. Aumento de datos	28
3.6. Parada temprana	29
3.7. Regularización por <i>dropout</i>	30
3.8. Tokenización	32
3.9. Bloque del transformador	35
4.1. Estructura de una CNN	40
4.2. Arquitectura ViT	41
4.3. Entrenamiento del modelo CLIP	44
4.4. Uso de CLIP para predicción <i>zero-shot</i>	46
5.1. Distribución del precio de la vivienda en Gijón	50
5.2. Análisis de la variable superficie	52
5.3. Distribución de precios en Gijón por zonas	53
5.4. Impacto de las características estructurales en el precio	54
5.5. Puntuaciones visuales de CLIP por vivienda	57
6.1. Diagnóstico de errores del modelo M3	64
6.2. Interpretabilidad modelo M3	65
6.3. Anomalías predictivas del modelo M3	66
6.4. Evaluación de residuos del Modelo M6	68

Índice de siglas

ANN	Artificial Neural Network	MAE	Mean Absolute Error
CLIP	Contrastive Language-Image Pre-training	MAPE	Mean Absolute Percentage Error
CNN	Convolutional Neural Network	ML	Machine Learning
DL	Deep Learning	MLP	Multilayer Perceptron
FE	Feature Engineering	MSE	Mean Squared Error
IA	Inteligencia Artificial	NLP	Natural Language Processing
ILSVRC	ImageNet Large Scale Visual Recognition Challenge	PCA	Principal Component Analysis
IQR	Interquartile Range	ReLU	Rectified Linear Unit
LLM	Large Language Model	RMSE	Root Mean Squared Error
LOWESS	Locally Weighted Scatterplot Smoothing	RNN	Recurrent Neural Network
		SGD	Stochastic Gradient Descent
		ViT	Vision Transformer

1. Introducción y objetivos

En la última década, el acceso a la vivienda se ha consolidado como uno de los principales retos socioeconómicos en España. La escasez de suelo finalista, la falta de un parque público de vivienda asequible y el incremento en los costes de financiación han generado un desequilibrio entre la oferta y la demanda. En febrero de 2026, el mercado inmobiliario registra tensiones históricas, con incrementos interanuales en el precio de venta de hasta el 17,7% [1], en un contexto en el que el precio medio nacional ha superado los máximos históricos registrados antes de la crisis financiera de 2008 [2, 3].

Este escenario crea un mercado extremadamente volátil, en el que la falta de claridad en la valoración de los inmuebles termina afectando tanto a compradores como a vendedores. Esta situación acentúa la necesidad de poder estimar de forma objetiva el valor de una vivienda.

Como caso de estudio, este trabajo se centra en la ciudad de Gijón, que presenta un recalentamiento similar al del conjunto nacional, con incrementos interanuales en torno al 17-18% [4]. Gijón presenta además una morfología heterogénea, con diferencias de precio notables según la zona. Distritos como el Centro presentan precios cercanos a 3.300 €/m² mientras que Gijón Oeste o El Llano, en torno a 2.000 €/m² (ver Figura 1.1). Esta heterogeneidad convierte a Gijón en un caso de estudio adecuado para evaluar un sistema de valoración inmobiliaria.

La tasación inmobiliaria automatizada se ha apoyado tradicionalmente en algoritmos deterministas y reglas fijas [5], que ofrecen estimaciones a partir de variables tabulares como superficie, número de habitaciones o ubicación. No obstante, el precio de una vivienda depende también de factores visuales y estéticos, difícilmente codificables como variables estructuradas. Incorporar esta información puede aportar señales complementarias a las que ya recogen los modelos clásicos.

El objetivo principal de este trabajo es construir y evaluar un modelo multimodal de valoración inmobiliaria para viviendas en Gijón, combinando variables tabulares con puntuaciones visuales obtenidas mediante CLIP a partir de las fotografías de los anuncios. CLIP (*Contrastive Language-Image Pre-training*) es un modelo desarrollado por OpenAI que aprende, a partir de cientos de millones de pares (imagen, texto), un espacio de representación compartido en el que imágenes y descripciones semánticamente afines quedan próximas entre sí [6]. Esta capacidad permite utilizarlo en modo *zero-shot*: dado un par de descripciones polares (por ejemplo, *luminoso* frente a *oscuro*), el modelo cuantifica la similitud de cada fotografía con ambos extremos, produciendo una puntuación continua por atributo. De este modo, es posible extraer variables visuales interpretables como luminosidad, estado de conservación o calidad de los acabados, sin etiquetar manualmente ninguna imagen.

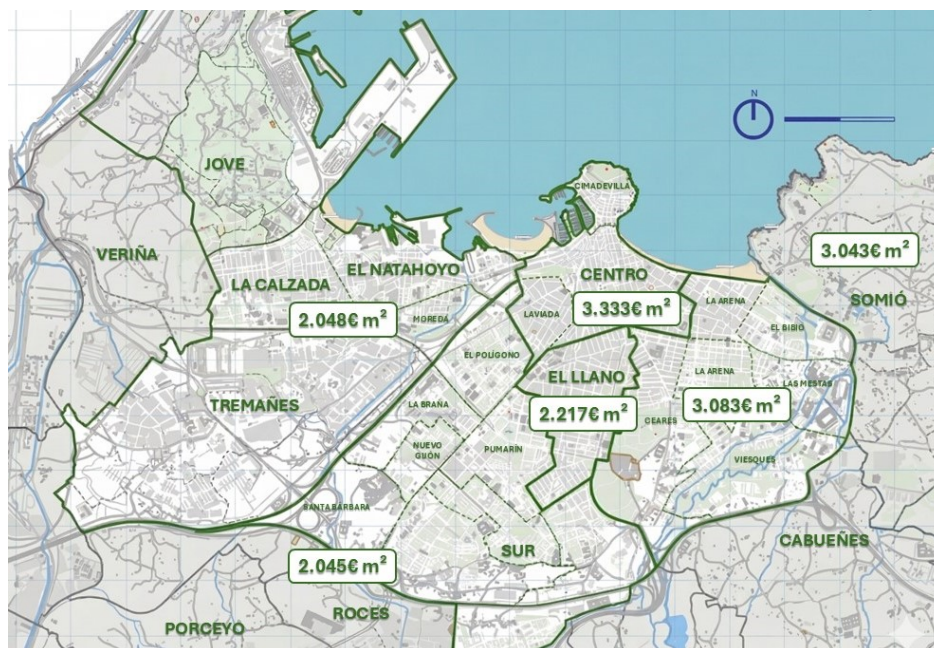


Figura 1.1 Mapa de Gijón por zonas. Gijón presenta una morfología heterogénea, con brechas de precio según la zona. Distritos como el Centro presentan precios por metro cuadrado muy superiores a El Llano o Gijón Oeste. Mapa extraído del [Callejero de Gijón](#) y completado con datos procedentes de [4].

Posteriormente, el rendimiento del modelo se comparará con un modelo *baseline* que únicamente utiliza datos tabulares. Esta base servirá como una referencia sencilla e interpretable, superior a la predicción aleatoria, sobre la que medir la aportación real del componente visual. Los objetivos específicos son:

- Formalizar el problema como una tarea de regresión supervisada y desarrollar el marco estadístico que justifica el uso de redes neuronales y arquitecturas multimodales.
- Construir un *baseline* tabular que sirva de referencia comparativa.
- Extraer puntuaciones visuales mediante CLIP e integrarlas con las variables estructuradas.
- Comparar el rendimiento del modelo multimodal frente al *baseline* y discutir sus limitaciones.

2. Fundamentos del aprendizaje automático

2.1. Inteligencia artificial

La **inteligencia artificial** (IA) carece de una definición académica universalmente aceptada. Históricamente, el concepto ha funcionado como un término paraguas que agrupa enfoques dispares, generando descripciones que a menudo son incompatibles entre sí. En este trabajo se adopta la formulación propuesta por Wang [7], que entiende la IA como «la capacidad de un sistema de procesamiento de información para operar bajo la restricción de conocimiento y recursos insuficientes». Bajo esta definición, un sistema inteligente no se caracteriza por ejecutar cálculos complejos, sino por operar en entornos inciertos ajustando su comportamiento a la información disponible.

Dentro de este marco, el **aprendizaje automático** (ML, *Machine Learning*) constituye el subconjunto de la IA dedicado al desarrollo de algoritmos que ajustan modelos matemáticos a partir de datos observados. Su objetivo es identificar patrones que permitan realizar predicciones o tomar decisiones sobre datos no vistos previamente. En función de la información disponible durante el entrenamiento, el aprendizaje automático se clasifica habitualmente en aprendizaje supervisado, no supervisado y por refuerzo. A su vez, el **aprendizaje profundo** (DL, *Deep Learning*) es la rama del ML basada en redes neuronales profundas, especialmente relevante en problemas de percepción, visión artificial y procesamiento de lenguaje.

2.1.1. Aprendizaje supervisado

El aprendizaje supervisado es una de las formas de ML más utilizadas en la práctica. Su objetivo es aprender un mapeo de unos datos de entrada $\mathbf{x}$ a unos datos de salida y , dado un conjunto etiquetado de pares entrada-salida $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. Aquí $\mathcal{D}$ es el conjunto de datos de entrenamiento y N es el número de muestras de entrenamiento. En función de la naturaleza de la variable de respuesta y , se ajusta la tarea del problema. Dos de las categorías principales son [8, 9]:

- **Clasificación.** El programa debe determinar a qué categoría de un conjunto dado pertenecen los datos de entrada. En este caso $y \in \{1, \dots, k\}$, donde k es el número de clases. Cuando $k = 2$ el problema recibe el nombre de **clasificación binaria**, mientras que se denomina **clasificación multiclase** siempre que $k > 2$. Un ejemplo de clasificación es un filtro de spam para e-mail en base al cuerpo del correo.

- **Regresión.** El modelo trata de predecir una salida numérica ($y \in \mathbb{R}$) a partir de los datos de entrada. Un ejemplo de regresión es predecir la temperatura que hará mañana utilizando otros parámetros meteorológicos.

2.1.2. Aprendizaje no supervisado

En el aprendizaje no supervisado no existen datos de salida, es decir, $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$ y el objetivo es encontrar patrones en los datos de entrada. Esta tarea se formaliza como una estimación de densidad, buscando modelos de la forma $p(\mathbf{x}_i|\boldsymbol{\theta})$ [9]. El aprendizaje no supervisado se puede aplicar en un conjunto de escenarios más amplio que el supervisado, ya que no requiere que un experto humano etiquete manualmente los datos. Algunos de los ejemplos canónicos son:

- **Descubrimiento de clústeres.** Consiste en estimar el número de agrupaciones K mediante la distribución $p(K|\mathcal{D})$. Se busca asignar cada punto i a un clúster z_i (habitualmente denominado variable **latente**) calculando la probabilidad más alta: $z_i^* = \underset{k}{\operatorname{argmax}} p(z_i = k|x_i, \mathcal{D})$.
- **Factores latentes y reducción de dimensionalidad.** Cuando los datos tienen muchas dimensiones, se busca proyectarlos a un espacio de dimensión inferior conservando el máximo de información. El método más representativo es el **análisis de componentes principales** (PCA, *Principal Component Analysis*).
- **Estructura de grafos.** Se busca descubrir dependencias directas entre variables correlacionadas, representándolas mediante un grafo $\mathcal{G}$. Se aplica en el estudio de redes de proteínas en biología y en gestión de carteras de acciones [9].

2.1.3. Aprendizaje por refuerzo

El aprendizaje por refuerzo es un paradigma en el que un **agente** vive en un mundo con el que interactúa mediante la realización de acciones en cada paso temporal. Las acciones producen recompensas y el objetivo del agente es elegir las acciones que le proporcionan mayores recompensas en promedio [10]. Este tipo de aprendizaje presenta dos dificultades características:

- **Asignación de crédito temporal.** Surge cuando la recompensa se recibe tiempo después de las acciones que la originaron, lo que requiere que el sistema identifique qué decisiones específicas fueron determinantes para el éxito.
- **Dilema exploración-explotación.** El agente debe decidir constantemente entre explotar estrategias conocidas que ya reportan beneficios o explorar nuevas acciones para descubrir soluciones potencialmente óptimas.

Dos ejemplos ilustrativos son la locomoción robótica, donde un agente debe coordinar múltiples articulaciones para alcanzar un objetivo; y juegos de estrategia como el ajedrez, donde

cada movimiento altera el estado del sistema de forma no determinista en función de la respuesta del oponente.

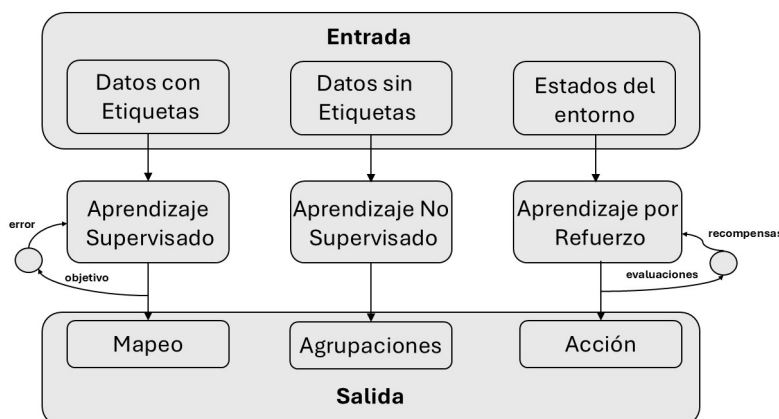


Figura 2.1 Comparación entre aprendizaje supervisado, no supervisado y por refuerzo. El primero aprende un mapeo a partir de datos etiquetados; el segundo identifica estructura en datos no etiquetados; y el tercero optimiza las acciones de un agente mediante recompensas obtenidas al interactuar con un entorno. Imagen adaptada de [11].

2.2. Conceptos fundamentales del aprendizaje automático

2.2.1. Generalización, sobreajuste y subajuste

El desafío central del aprendizaje automático consiste en obtener un buen rendimiento en entradas nunca vistas, no en aquellas en las que el modelo fue entrenado. La capacidad de un modelo de ML de generar buenas predicciones en datos no observados se conoce como **generalización**.

Durante el entrenamiento, el proceso se reduce a minimizar el error sobre el conjunto de datos disponible. El objetivo real del aprendizaje automático, sin embargo, es minimizar el **error de generalización**: el error del modelo sobre datos no vistos. La dificultad se encuentra en que este error no se puede medir directamente, pues el comportamiento del modelo solo se observa sobre los datos de entrenamiento.

Para solucionar este problema, se asume que los datos de entrenamiento y prueba son generados por un proceso de generación de datos probabilístico. Nos basamos en la **suposición de variables independientes e idénticamente distribuidas** (i.i.d.). Llamamos a la distribución subyacente compartida **distribución generadora de datos**: p_{data} . Bajo este proceso, el valor esperado del error de prueba es mayor o igual que el error esperado de entrenamiento. Esto da lugar a dos patologías principales en el aprendizaje de modelos:

- **Subajuste** (*underfitting*). Incapacidad del modelo para reducir suficientemente el error de entrenamiento debido a una excesiva simplicidad.
- **Sobreajuste** (*overfitting*). Ocurre cuando el error de entrenamiento es bajo, pero la brecha con el error de prueba es excesiva; el modelo «memoriza» el ruido en lugar de aprender el patrón.

La dualidad entre el sobreajuste y el subajuste es uno de los desafíos centrales del aprendizaje automático. La Figura 2.2 muestra la comparativa entre ambos y un modelo bien ajustado.

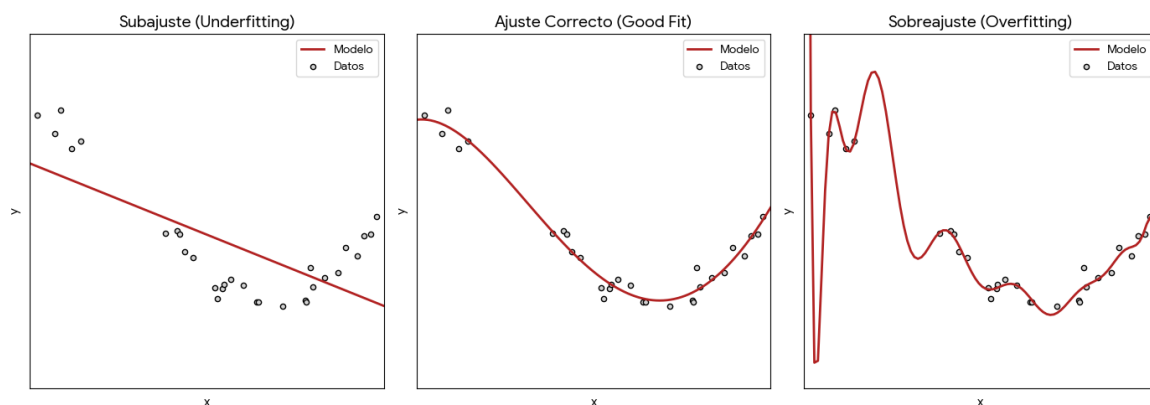


Figura 2.2 Comparación entre subajuste, buen ajuste y sobreajuste. El subajuste aparece cuando el modelo es demasiado simple para capturar la tendencia de los datos; el buen ajuste aproxima la estructura subyacente sin reproducir el ruido; y el sobreajuste surge cuando el modelo se adapta en exceso a la muestra de entrenamiento, perdiendo capacidad de generalización.

2.2.2. Estimadores, sesgo y varianza

La estadística proporciona las herramientas base del aprendizaje automático. Conceptos como la estimación de parámetros, el sesgo y la varianza son útiles para caracterizar formalmente la generalización, el subajuste y el sobreajuste.

Estimación puntual

Los estimadores puntuales pretenden ofrecer un valor único, es decir, un punto en la recta real, que aproxime un parámetro poblacional de interés. Sea $\{x_i\}_{i=1}^N$ un conjunto de N muestras i.i.d., un **estimador puntual** o **estadístico** es cualquier función de los datos:

$$\hat{\theta}_N = g(x_1, \dots, x_N). \quad (2.1)$$

Esta definición no requiere que g devuelva un valor cercano al verdadero θ , ni siquiera dentro de su rango de valores. Se trata de una definición muy general que permite el desarrollo de estimadores de una forma muy flexible [8].

Desde la perspectiva de la interpretación frecuentista de la probabilidad, el valor de θ es fijo

pero desconocido. Dado que los datos provienen de un proceso aleatorio, cualquier función de los mismos es aleatoria. Así, $\hat{\theta}$ es una variable aleatoria.

Sesgo

El **sesgo** de un estimador se define como la diferencia entre la esperanza y el valor verdadero del parámetro:

$$\text{Sesgo}(\hat{\theta}_N) = \mathbb{E}[\hat{\theta}_N] - \theta. \quad (2.2)$$

Un estimador es **insesgado** si su sesgo es 0 para cualquier tamaño de la muestra. Por otro lado, un estimador es **asintóticamente insesgado** si su valor tiende al parámetro cuando el tamaño muestral tiende a infinito:

$$\lim_{N \rightarrow \infty} \mathbb{E}[\hat{\theta}_N] = \theta \quad (2.3)$$

En términos prácticos, esto implica que el sesgo del estimador tiende a cero conforme aumenta el volumen de datos observados. Esta es una condición más débil que la insesgadez estricta, pero útil en aprendizaje automático, donde habitualmente se trabaja con grandes conjuntos de datos. No obstante, la reducción del sesgo depende de que el modelo y el procedimiento de estimación sean adecuados.

Varianza, desviación típica y error estándar

La **varianza** de un estimador cuantifica la dispersión de la variable aleatoria $\hat{\theta}_N$ alrededor de su valor esperado $\mathbb{E}[\hat{\theta}_N]$, evaluada sobre todas las posibles muestras i.i.d. de tamaño N . Se define como la esperanza del cuadrado de la desviación del estimador respecto a su media:

$$\text{Var}(\hat{\theta}_N) = \mathbb{E} \left[\left(\hat{\theta}_N - \mathbb{E}[\hat{\theta}_N] \right)^2 \right]. \quad (2.4)$$

La raíz cuadrada de la varianza de un estimador se denomina **error estándar** y se denota $\text{SE}(\hat{\theta})$. El error estándar se diferencia de la **desviación típica** poblacional σ , que mide la dispersión de los datos originales, no del estimador.

A menudo, el error estándar depende de la varianza poblacional σ^2 , que es desconocida, por lo que es necesario estimarla. Un primer estimador de la varianza poblacional σ^2 es la varianza muestral:

$$\hat{\sigma}_N^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu}_N)^2, \quad (2.5)$$

donde $\hat{\mu}_N$ es la media muestral. Sin embargo, la esperanza de este estimador es $\mathbb{E}[\hat{\sigma}_N^2] = \frac{N-1}{N} \sigma^2$, subestimando sistemáticamente la varianza. Para corregir esto, se utiliza el estimador

insesgado de la varianza muestral, que divide por $N - 1$ en lugar de N :

$$\tilde{\sigma}_N^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \hat{\mu}_N)^2. \quad (2.6)$$

La linealidad de la esperanza confirma que este estimador no tiene sesgo:

$$\mathbb{E}[\tilde{\sigma}_N^2] = \frac{N}{N-1} \mathbb{E}[\hat{\sigma}_N^2] = \frac{N}{N-1} \left(\frac{N-1}{N} \sigma^2 \right) = \sigma^2. \quad (2.7)$$

No obstante, debido a que la función $f(x) = \sqrt{x}$ es estrictamente cóncava, la **Desigualdad de Jensen** establece que ni la raíz cuadrada de la varianza muestral ni la raíz cuadrada del estimador insesgado de la varianza proporcionan estimadores insesgados de la desviación típica poblacional.

Lema 2.1 (Desigualdad de Jensen). *Sea X una variable aleatoria no constante y sea $f(x)$ una función estrictamente cóncava en el dominio de X . Si las esperanzas matemáticas existen, entonces el valor esperado de la función de la variable aleatoria es estrictamente menor que la función del valor esperado de la variable aleatoria ¹:*

$$\mathbb{E}[f(X)] < f(\mathbb{E}[X]). \quad (2.8)$$

Proposición 2.1. *Sea σ^2 la varianza poblacional y N el tamaño muestral. Los siguientes estimadores de la desviación típica σ son sesgados:*

1. $\sqrt{\tilde{\sigma}_N^2} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \hat{\mu}_N)^2}$
2. $\sqrt{\hat{\sigma}_N^2} = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu}_N)^2}$

Demostración. Consideremos la función de transformación $f(x) = \sqrt{x}$. Evaluando la segunda derivada de esta función para todo $x > 0$, obtenemos:

$$f''(x) = -\frac{1}{4x^{3/2}}. \quad (2.9)$$

Dado que $f''(x) < 0$ para todo $x > 0$, la función $f(x) = \sqrt{x}$ es estrictamente cóncava. Se mostró que $\tilde{\sigma}_N^2$ es un estimador insesgado de la varianza, de modo que su valor esperado es exactamente σ^2 .

Aplicando el Lema 2.1, tomando como variable aleatoria $X = \tilde{\sigma}_N^2$ y como función estrictamente cóncava $f(x) = \sqrt{x}$, obtenemos la siguiente relación:

$$\mathbb{E} \left[\sqrt{\tilde{\sigma}_N^2} \right] < \sqrt{\mathbb{E} [\tilde{\sigma}_N^2]} \quad (2.10)$$

¹Se puede encontrar una demostración de esta desigualdad en el capítulo 5 del libro de Billingsley [12].

Sustituyendo el valor esperado de la varianza en el lado derecho de la inecuación, resulta:

$$\mathbb{E} \left[\sqrt{\hat{\sigma}_N^2} \right] < \sqrt{\sigma^2} = \sigma \quad (2.11)$$

Por otro lado, si consideramos el estimador sesgado $\hat{\sigma}_N^2$, se tiene que

$$\mathbb{E}[\hat{\sigma}_N^2] = \frac{N-1}{N} \sigma^2 < \sigma^2, \quad (2.12)$$

y, aplicando Jensen de nuevo:

$$\mathbb{E} \left[\sqrt{\hat{\sigma}_N^2} \right] < \sqrt{\mathbb{E}[\hat{\sigma}_N^2]} = \sqrt{\frac{N-1}{N} \sigma^2} < \sigma. \quad (2.13)$$

□

Compromiso sesgo-varianza

El sesgo y la varianza miden dos fuentes de error en un estimador. El sesgo mide la desviación esperada respecto al valor real del parámetro. La varianza, por otro lado, proporciona una medida de la desviación con respecto al valor esperado del estimador que puede causar cualquier muestreo particular de los datos.

El problema surge al encontrarnos con dos estimadores, uno con mayor sesgo y otro con mayor varianza. La forma más común de negociar este compromiso es mediante el **error cuadrático medio** (MSE, *Mean Squared Error*) de las estimaciones [8]:

$$\text{MSE} = \mathbb{E}[(\hat{\theta}_N - \theta)^2] = \text{Sesgo}(\hat{\theta}_N)^2 + \text{Var}(\hat{\theta}_N). \quad (2.14)$$

La evaluación del MSE incorpora tanto al sesgo como a la varianza. Los estimadores deseables son aquellos con un MSE pequeño, y estos son los que logran mantener tanto su sesgo como su varianza controlados.

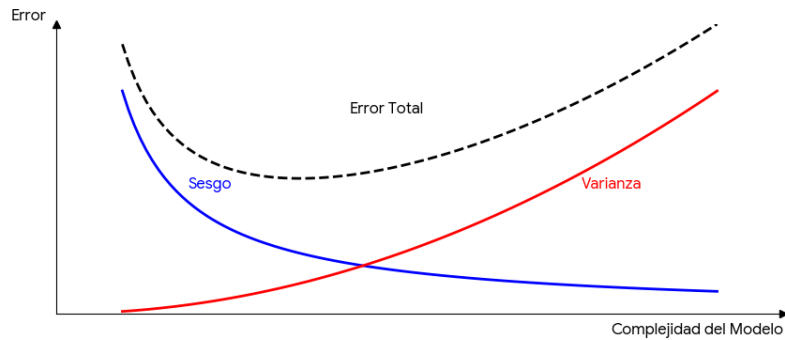


Figura 2.3 Compromiso sesgo-varianza. El modelo óptimo es aquel capaz de minimizar ambas magnitudes a la vez. Esto es equivalente a minimizar el MSE.

2.2.3. Hiperparámetros y validación

En el desarrollo de algoritmos de ML, resulta esencial diferenciar entre los parámetros internos del modelo, que se ajustan de forma automática durante el entrenamiento, y los **hiperparámetros**, configuraciones externas que regulan el comportamiento y la capacidad del algoritmo [8]. Ejemplos típicos de estos últimos son el grado de un polinomio en problemas de regresión o la cantidad de vecinos K en los métodos de K vecinos cercanos [9].

Conjuntos de entrenamiento, validación y prueba

Para estimar el error de generalización y seleccionar hiperparámetros sin sesgar la evaluación final, es fundamental separar los datos disponibles en subconjuntos disjuntos. De forma estándar, se aplica una estrategia de **división simple**, en donde el conjunto de datos se divide en tres subconjuntos:

- **Conjunto de entrenamiento.** Utilizado para el ajuste de los parámetros del modelo.
- **Conjunto de validación.** Un subconjunto de los datos de entrenamiento (típicamente un 20 %) reservado exclusivamente para guiar la selección de hiperparámetros. El error calculado se utiliza como aproximación del error de generalización.
- **Conjunto de prueba.** Un conjunto totalmente independiente que no debe influir en la construcción o selección del modelo. Su único propósito es proporcionar una estimación final e imparcial del rendimiento del sistema una vez completado el proceso de aprendizaje.

Validación cruzada

Cuando el tamaño del conjunto de datos es reducido, la división simple presenta una serie de problemas estadísticos: reduce los datos disponibles para el entrenamiento y aumenta la varianza en la estimación del error debido al pequeño tamaño del conjunto de validación. Para mitigar esto, se emplea la técnica de **validación cruzada** con K iteraciones, que funciona de la siguiente manera:

1. Particionar el conjunto de datos en K subconjuntos no superpuestos.
2. Realizar K ensayos independientes, donde en cada iteración i , el subconjunto i -ésimo se utiliza para validación y los $K - 1$ restantes para entrenamiento.
3. Estimar el error de generalización promediando los errores obtenidos en las K pruebas.

2.2.4. Estimación de máxima verosimilitud

En secciones previas se discutió el concepto de estimador y sus propiedades principales. No obstante, no se discutió cómo obtener estos estimadores. Una de las técnicas más utilizadas es el **principio de máxima verosimilitud**. Dado un conjunto de N datos $X = \{x_i\}_{i=1}^N$

extraídos independientemente de una distribución real y desconocida $p_{\text{data}}(x)$, se define una familia paramétrica de distribuciones $p_{\text{modelo}}(x; \theta)$. El objetivo es encontrar los parámetros θ que maximicen la probabilidad de observar los datos de entrenamiento bajo nuestro modelo:

$$\hat{\theta}_{ML} = \arg \max_{\theta} \prod_{i=1}^N p_{\text{modelo}}(x_i; \theta). \quad (2.15)$$

Desde un punto de vista numérico esta ecuación puede dar lugar a **subdesbordamiento aritmético**. Al multiplicar muchos valores numéricos entre 0 y 1 se obtienen números muy pequeños imposibles de representar con precisión finita. Por tanto, aprovechando que el logaritmo es una función monótona, se puede estimar θ como

$$\hat{\theta}_{ML} = \arg \max_{\theta} \sum_{i=1}^N \log p_{\text{modelo}}(x_i; \theta). \quad (2.16)$$

Una forma de interpretar la estimación de máxima verosimilitud es verla como minimizar la disimilitud entre la distribución empírica y la distribución del modelo. El grado de disimilitud queda determinado por la **divergencia de Kullback-Leibler** [8]:

$$D_{KL}(\hat{p}_{\text{data}} \parallel p_{\text{modelo}}) = \mathbb{E}_{x \sim \hat{p}_{\text{data}}} [\log \hat{p}_{\text{data}}(x) - \log p_{\text{modelo}}(x; \theta)] \quad (2.17)$$

El primer término de la derecha es una función solo del proceso de generación de los datos, no del modelo. Por tanto, si entrenamos el modelo para minimizar la divergencia KL sólo tenemos que minimizar el término:

$$\mathbb{E}_{x \sim \hat{p}_{\text{data}}} [-\log p_{\text{modelo}}(x; \theta)]. \quad (2.18)$$

Esta perspectiva resulta más apropiada pues una gran parte de los algoritmos de optimización numérica están diseñados para la búsqueda de mínimos, no máximos [8].

2.2.5. Desafíos

En esta sección discutiremos algunos de los desafíos fundamentales a los que se enfrenta el aprendizaje automático.

La maldición de la dimensionalidad

Muchos problemas de ML se vuelven más difíciles al aumentar la dimensión del espacio de características. Este fenómeno, conocido como **maldición de la dimensionalidad**, se debe a que el número de configuraciones posibles crece exponencialmente con el número de variables. Si cada dimensión se divide en v intervalos, un espacio de dimensión d requiere $O(v^d)$ celdas, por lo que los datos disponibles ocupan una fracción cada vez menor del espacio [10].

Como consecuencia, el espacio de características se vuelve disperso y la mayoría de regiones no contiene ejemplos de entrenamiento. Esto perjudica la generalización, ya que muchos métodos

de aprendizaje se apoyan en la similitud local: un nuevo punto se predice a partir de ejemplos cercanos. En alta dimensión, esa cercanía pierde utilidad porque es probable que el punto caiga en una región sin observaciones próximas [10].

El Teorema de «*No Free Lunch*»

Una gran parte del aprendizaje automático se centra en diseñar diferentes modelos y elegir el mejor de ellos para cada problema particular, a través de técnicas como la validación cruzada. No obstante, no existe un modelo universal. Esto es, un modelo que ofrezca el mejor rendimiento en todos los problemas dados. El teorema *No Free Lunch* (no hay almuerzo gratis), propuesto por Wolpert [13], establece que, si se promedia el rendimiento sobre todas las posibles distribuciones generadoras de datos, cualquier algoritmo de clasificación obtendrá exactamente la misma tasa de error al evaluar puntos no observados previamente. La razón de esto es que el conjunto de asunciones útiles para un modelo pueden ser insuficientes para otro.

Las consecuencias prácticas de este teorema son la necesidad de considerar distintas arquitecturas y la importancia de la validación empírica. Dado que ningún modelo es óptimo para todos los problemas, resulta necesario comparar familias de modelos capaces de capturar estructuras diferentes en los datos. Además, como no puede determinarse de antemano qué modelo será el más adecuado para un fenómeno concreto, la selección debe apoyarse en procedimientos como la validación cruzada y la evaluación sobre conjuntos de prueba independientes.

3. Redes Neuronales

Las redes neuronales artificiales (ANN, *Artificial Neural Networks*) son modelos computacionales inspirados en el mecanismo de las neuronas biológicas. Fueron introducidas por McCulloch y Pitts [14] como un modelo simplificado que trataba de emular su comportamiento.

Las redes neuronales modernas son redes de pequeñas unidades de procesamiento conectadas entre sí. En esta sección introducimos la **red neuronal prealimentada** o *feedforward network*, llamada así debido a que la computación ocurre de forma iterativa de una capa de unidades a la siguiente. El adjetivo profundo hace referencia a la arquitectura de estos modelos, la cual se caracteriza por la concatenación de múltiples capas ocultas sucesivas.

3.1. La unidad

El componente básico de una red neuronal es una unidad computacional individual. Una unidad toma un conjunto de valores como entrada, realiza una operación sobre estos y produce una salida:

$$z = b + \sum_{i=1}^n w_i x_i, \quad (3.1)$$

donde $x_1 \dots x_n$ son las entradas, $w_1 \dots w_n$ son los **pesos** y b es el **sesgo**. Resulta conveniente introducir $\mathbf{x} = [x_1, \dots, x_n]^\top$ y $\mathbf{w} = [w_1, \dots, w_n]^\top$ dando lugar a una expresión equivalente como producto escalar:

$$z = \mathbf{w}^\top \mathbf{x} + b. \quad (3.2)$$

Esta expresión representa una transformación afín. Esto supone una grave limitación a la capacidad de una unidad para aprender el comportamiento de datos no lineales. Por ello, se introduce una **función de activación** f que se aplica sobre la ecuación anterior. Así, la salida y de la neurona se calcula como:

$$y = f(z) = f(\mathbf{w}^\top \mathbf{x} + b). \quad (3.3)$$

Algunas de las funciones de activación más relevantes son:

- **Sigmoide.** Se define como $\sigma(z) = \frac{1}{1+e^{-z}}$. Es una función diferenciable y mapea la salida al intervalo $(0, 1)$. Esto es de especial utilidad pues comprime los valores atípicos dentro de un rango acotado.
- **Tangente hiperbólica.** Una función similar a la sigmoide utilizada con mayor frecuencia $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Es una variación de la sigmoide que toma valores entre -1 y 1.

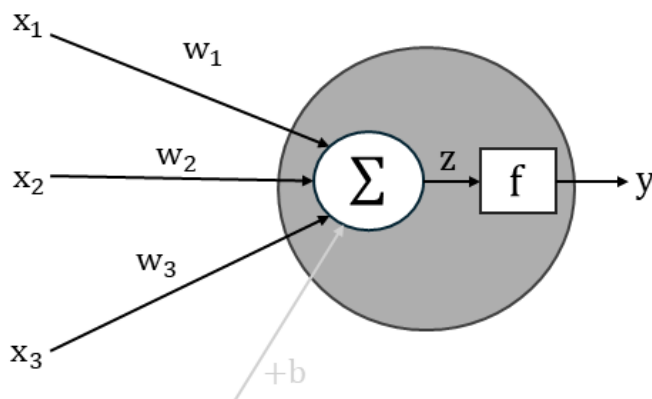


Figura 3.1 Una neurona tomando 3 entradas x_1, x_2, x_3 y produciendo una salida y . Para poder aprender el comportamiento de datos no lineales, se introducen las funciones de activación.

- **ReLU** (*Rectified Linear Unit*). Definida como $\text{ReLU}(z) = \max(z, 0)$ tiene propiedades distintas de las anteriores debido a su similitud con una función lineal. Las funciones sigmoide y tanh tienen problemas de saturación pues valores muy elevados de $|z|$ tienen derivada casi nula. Este problema se conoce como el **desvanecimiento del gradiente**.

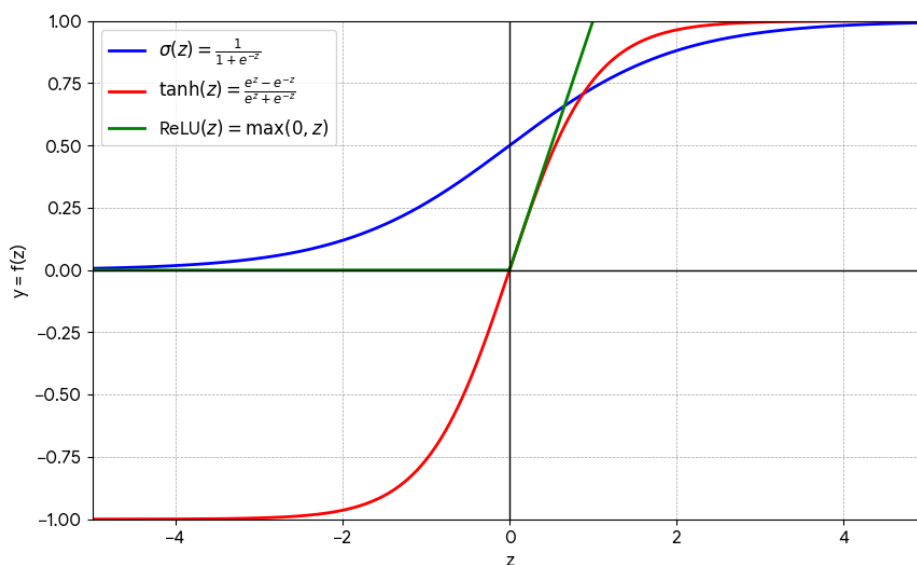


Figura 3.2 Representación de tres de las funciones de activación más utilizadas en la práctica. La sigmoide y la tangente hiperbólica son similares en diferenciabilidad y comprimen la salida a un intervalo acotado; la ReLU, lineal a trozos, evita la saturación del gradiente.

3.2. El perceptrón multicapa

El **perceptrón multicapa** (MLP, *Multilayer Perceptron*) es uno de los tipos más sencillos de red neuronal en el que las neuronas están conectadas sin ciclos. Esto implica que las salidas de las unidades en cada capa se introducen como entradas a la siguiente capa.

Las redes MLP están formadas por tres tipos de nodos: **unidades de entrada**, **unidades ocultas** y **unidades de salida** (ver Figura 3.3). La capa de entrada se representa mediante un vector $\mathbf{x}$, mientras que la capa de salida depende de la naturaleza del problema. En tareas de regresión, la salida es un escalar $y \in \mathbb{R}$, mientras que en problemas de clasificación se dispone de k neuronas de salida $y_1, \dots, y_k$, correspondientes a las distintas clases.

El núcleo de la red lo constituyen las capas ocultas $\mathbf{x}^{[l]}$, formadas por unidades $\mathbf{x}_j^{[l]}$, con $j = 1, \dots, n_l$, donde n_l denota el número de neuronas en la capa l . De acuerdo con la Ecuación 3.2, cada unidad está parametrizada por un conjunto de pesos y un sesgo. A nivel de capa, estos parámetros se agrupan en una matriz de pesos $\mathbf{W}^{[l]}$ y un vector de sesgos $\mathbf{b}^{[l]}$. De forma compacta, el conjunto total de parámetros del modelo se denota por

$$\boldsymbol{\theta} = \{\mathbf{W}^{[l]}, \mathbf{b}^{[l]}\}_{l=1}^L, \quad (3.4)$$

donde L es el número de capas de la red. Cada elemento $\mathbf{W}_{ji}^{[l]}$ representa el peso asociado a la conexión entre la i -ésima unidad de la capa $l-1$ y la j -ésima unidad de la capa l . La computación en cada capa consiste en tres operaciones: una transformación lineal $\mathbf{W}^{[l]}\mathbf{x}^{[l-1]}$, la suma del sesgo $\mathbf{b}^{[l]}$, y la aplicación de una función de activación $f^{[l]}$. Por convención, la capa de entrada se denota como $\mathbf{x}^{[0]}$. A continuación, se muestra de forma sintetizada el algoritmo de predicción en un perceptrón multicapa:

Algoritmo 1: Propagación hacia adelante

Entrada: Entrada de la red $\mathbf{x}^{[0]}$

Salida: Predicción de la red $\hat{\mathbf{y}}$

```

1 para  $l = 1$  a  $L$  hacer
2    $\mathbf{z}^{[l]} = \mathbf{W}^{[l]}\mathbf{x}^{[l-1]} + \mathbf{b}^{[l]}$ ;
3    $\mathbf{x}^{[l]} = f^{[l]}(\mathbf{z}^{[l]})$ ;
4 fin
5  $\hat{\mathbf{y}} = \mathbf{x}^{[L]}$ ;
6 devolver  $\hat{\mathbf{y}}$ ;
```

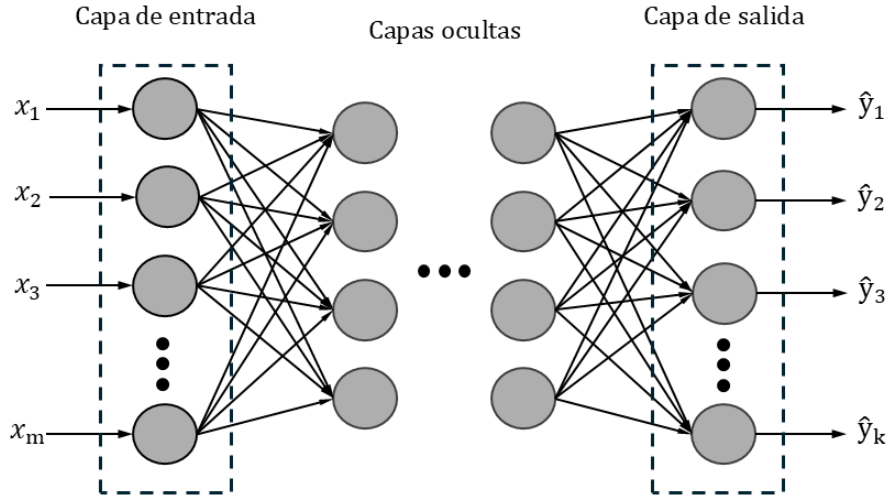


Figura 3.3 Estructura de un perceptrón multicapa con m unidades de entrada y k unidades de salida correspondientes a las distintas clases. Las capas ocultas intermedias transforman la entrada mediante combinaciones lineales y activaciones no lineales.

3.3. Entrenamiento de una red neuronal

El perceptrón multicapa se enmarca dentro del aprendizaje supervisado. El objetivo es ajustar los parámetros del modelo θ de forma que la predicción $\hat{y}$ aproxime adecuadamente el valor observado y .

Primero, será necesaria una **función coste**, capaz de modelar la distancia entre la salida del sistema y el valor real. Segundo, se necesitará un algoritmo de optimización capaz de encontrar los parámetros que minimicen esta función coste. Para ello se utiliza el **descenso del gradiente**. Este algoritmo requiere conocer el gradiente de la función coste, que contiene las derivadas parciales de esta función respecto a cada uno de los parámetros de la red. Para encontrar estos valores en todas las capas intermedias se utiliza el algoritmo de **retropropagación**.

3.3.1. Funciones coste

Uno de los aspectos más relevantes en el diseño de una red neuronal es la elección de la función coste. En la formulación probabilística del aprendizaje supervisado, se asume que, dada una entrada $\mathbf{x}$, la variable de salida y sigue una distribución condicional parametrizada $p_{\theta}(y | \mathbf{x})$. Bajo esta hipótesis, el entrenamiento del modelo puede plantearse mediante el principio de máxima verosimilitud detallado en la Sección 2.2.4. El objetivo es minimizar la log-verosimilitud negativa sobre la distribución empírica de los datos:

$$J(\theta) = -\mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{\text{data}}} [\log p_{\theta}(y | \mathbf{x})]. \quad (3.5)$$

Esta expresión proporciona un marco unificado para la elección de funciones de coste: su forma concreta depende de la familia de distribuciones asumida para $p_{\theta}(y \mid \mathbf{x})$. Por ejemplo, si se supone que $y \mid \mathbf{x}$ sigue una distribución gaussiana con media y varianza constante, la minimización de $J(\theta)$ conduce, salvo constantes, a una pérdida cuadrática equivalente al error cuadrático medio. En cambio, si $y \mid \mathbf{x}$ se modela mediante una distribución Bernoulli o categórica, la log-verosimilitud negativa da lugar a la **entropía cruzada**, ampliamente utilizada en problemas de clasificación [8, 15].

3.3.2. Algoritmos de optimización

Dado el conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, el objetivo del algoritmo de optimización es encontrar el conjunto de parámetros $\hat{\theta}$ que minimiza la función coste:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} J(\theta). \quad (3.6)$$

Existen muchas familias de algoritmos de optimización, pero los estándar para entrenar redes neuronales son iterativos. Estos algoritmos inician los parámetros de forma heurística y los ajustan de forma iterativa de modo que la función coste se reduce. El método más sencillo es el **descenso del gradiente** [8]:

Algoritmo 2: Descenso del Gradiente

Entrada: Tasa de aprendizaje η , parámetros actuales θ

Salida: Parámetros actualizados θ

- 1 Calcular las derivadas de J con respecto a θ : $\nabla_{\theta} J = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_p} \end{bmatrix}$;
 - 2 Actualizar los parámetros: $\theta \leftarrow \theta - \eta \nabla_{\theta} J$;
 - 3 **devolver** θ ;
-

El primer paso calcula el gradiente de la función pérdida en la posición actual, lo cual determina la dirección de máxima pendiente de esta. El segundo paso realiza un pequeño desplazamiento η , conocido como **tasa de aprendizaje**, en sentido descendente. Este parámetro puede ser fijo o bien se puede realizar una búsqueda de línea, donde se prueban varios valores de η para encontrar aquel que minimice en mayor medida la pérdida.

Al alcanzar un mínimo de la función coste, el gradiente se anula y los parámetros dejan de actualizarse. En la práctica, el entrenamiento suele detenerse cuando la norma del gradiente es suficientemente pequeña. Sin embargo, en superficies no convexas el gradiente puede anularse en mínimos locales o puntos de silla, por lo que el descenso del gradiente no garantiza la convergencia al mínimo global; su resultado depende de la inicialización y de la geometría de la función (ver Figura 3.4) [8].

Esta limitación es especialmente relevante en modelos de alta dimensión. El **descenso de gradiente estocástico** (SGD, *Stochastic Gradient Descent*) mitiga parcialmente este problema estimando el gradiente sobre subconjuntos aleatorios de datos, denominados *minibatches*, en lugar de usar todo el conjunto de entrenamiento. Esta aproximación introduce ruido en la dirección de actualización, permitiendo explorar regiones más amplias del espacio de parámetros y reduciendo la probabilidad de quedar atrapado prematuramente en soluciones subóptimas [8].

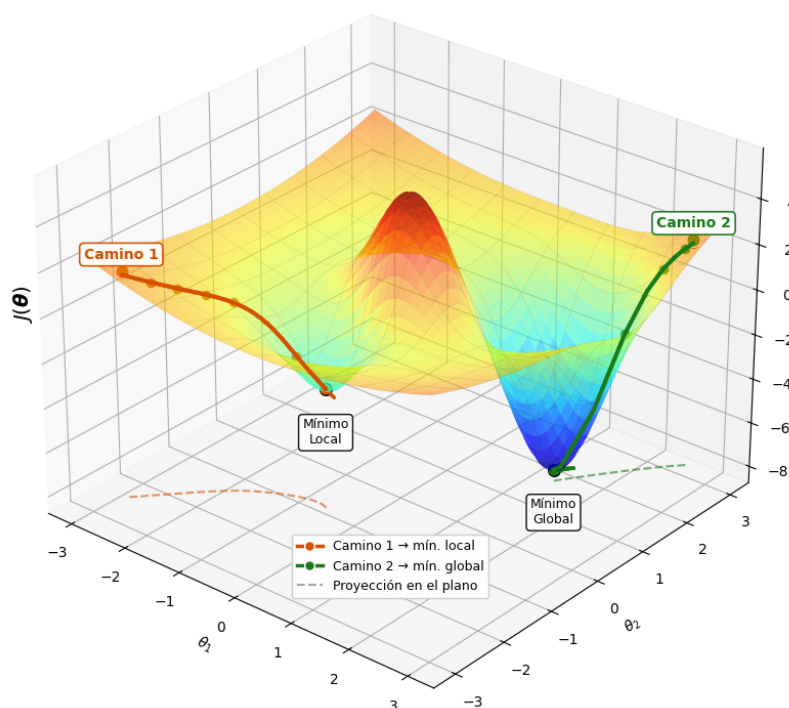


Figura 3.4 Esquema representativo algoritmo de optimización de una función coste en 2D. El algoritmo puede converger a un mínimo local o a un mínimo global en función de las condiciones iniciales.

3.3.3. Retropropagación

Hasta ahora hemos descrito cómo el algoritmo de optimización encuentra el conjunto de parámetros que minimiza la función coste. No obstante, este proceso requiere conocer el término $\nabla_{\theta} J(\theta)$ en cada iteración. El algoritmo de **retropropagación** popularizado en el contexto de las redes neuronales por Rumelhart, Hinton y Williams [16] computa este gradiente.

Este algoritmo está basado en la regla de la cadena, aplicando un cierto orden de operaciones que lo hacen altamente eficiente. Sean $x \in \mathbb{R}$, $f, g : \mathbb{R} \rightarrow \mathbb{R}$, $y = g(x)$ y $z = f(y)$, se tiene

que:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}. \quad (3.7)$$

Esta expresión se puede generalizar. En efecto, sean $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$, g una función de $\mathbb{R}^m$ a $\mathbb{R}^n$ y una función f de $\mathbb{R}^n$ a $\mathbb{R}$, la derivada de z respecto a una componente de x se define como:

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}. \quad (3.8)$$

En notación vectorial, esto se expresa de forma más compacta utilizando la matriz Jacobiana de g (denotada como $\frac{\partial y}{\partial x}$), que es una matriz de dimensiones $n \times m$ [8]:

$$\nabla_x z = \left(\frac{\partial y}{\partial x} \right)^\top \nabla_y z. \quad (3.9)$$

El algoritmo de retropropagación es una aplicación recursiva y estructurada de la regla de la cadena que se basa en dos etapas:

1. Pase hacia delante (*forward pass*). Ejecuta los datos de entrada a través de las ecuaciones de la red de principio a fin y almacena las activaciones resultantes de todas las unidades ocultas.
2. Pase hacia atrás (*backward pass*). Calcula las derivadas comenzando desde el final de la red y avanza en sentido inverso hacia la primera capa.

El Algoritmo 3 muestra el pase hacia adelante sobre una red MLP. La pérdida $L(\hat{y}, y)$ depende de la salida $\hat{y}$ y del objetivo y . Para obtener el coste total J , se puede sumar la pérdida a un regularizador $\Omega(\theta)$, concepto que definiremos con mayor detalle en la Sección 3.5. El Algoritmo 4 muestra cómo calcular los gradientes de J con respecto a los parámetros $\mathbf{W}$ y $\mathbf{b}$. Por simplicidad, esta demostración utiliza únicamente un solo ejemplo de entrada $\mathbf{x}$. En la práctica, el algoritmo se ejecuta sobre minilotes, no sobre el conjunto total de datos [8].

Algoritmo 3: Propagación hacia adelante

Entrada: Profundidad de la red, L **Entrada:** $\mathbf{W}^{[l]}, l \in \{1, \dots, L\}$, las matrices de pesos del modelo**Entrada:** $\mathbf{b}^{[l]}, l \in \{1, \dots, L\}$, los vectores de sesgo del modelo**Entrada:** $\mathbf{x}$, el vector de entrada a procesar**Entrada:** y , la salida objetivo

```

1  $\mathbf{h}^{[0]} = \mathbf{x}$ ;
2 para  $k = 1, \dots, L$  hacer
3    $\mathbf{z}^{[k]} = \mathbf{W}^{[k]}\mathbf{h}^{[k-1]} + \mathbf{b}^{[k]}$ ;
4    $\mathbf{h}^{[k]} = f(\mathbf{z}^{[k]})$ ;
5 fin
6  $\hat{y} = \mathbf{h}^{[L]}$ ;
7  $J = L(\hat{y}, y) + \lambda\Omega(\boldsymbol{\theta})$ ;
```

Algoritmo 4: Propagación hacia atrás

```

1 Después de la propagación hacia adelante, calcular el gradiente en la capa de salida;;
2  $\mathbf{g} \leftarrow \nabla_{\hat{\mathbf{y}}} J = \nabla_{\hat{\mathbf{y}}} L(\hat{\mathbf{y}}, y)$ ;
3 para  $k = L, L - 1, \dots, 1$  hacer
4   Convertir el gradiente de la salida de la capa en un gradiente de la activación pre-no
     linealidad;
5    $\mathbf{g} \leftarrow \nabla_{\mathbf{z}^{[k]}} J = \mathbf{g} \odot f'(\mathbf{z}^{[k]})$ ;
6   Calcular gradientes sobre pesos y vectores de sesgo (incluyendo el término de
     regularización, si lo hay);
7    $\nabla_{\mathbf{b}^{[k]}} J = \mathbf{g} + \lambda \nabla_{\mathbf{b}^{[k]}} \Omega(\boldsymbol{\theta})$ ;
8    $\nabla_{\mathbf{W}^{[k]}} J = \mathbf{g}(\mathbf{h}^{[k-1]})^\top + \lambda \nabla_{\mathbf{W}^{[k]}} \Omega(\boldsymbol{\theta})$ ;
9   Propagar los gradientes con respecto a las activaciones de la capa oculta del nivel
     inferior;
10   $\mathbf{g} \leftarrow \nabla_{\mathbf{h}^{[k-1]}} J = (\mathbf{W}^{[k]})^\top \mathbf{g}$ ;
11 fin
```

3.4. Teorema de aproximación universal

Las redes neuronales prealimentadas ofrecen un entorno idóneo para modelizar dependencias matemáticas entre entradas y salidas, independientemente de su complejidad. En particular, el **Teorema de aproximación universal**, demostrado por Hornik, Stinchcombe y White [17], formaliza esta idea. Establece que una red prealimentada con una capa de salida lineal y

al menos una capa oculta con activación no lineal puede aproximar cualquier función Borel-medible con precisión arbitraria, siempre que disponga de suficientes neuronas ocultas. El concepto de medibilidad Borel no se encuentra en los objetivos de este trabajo, pero basta decir que cualquier función continua definida sobre un conjunto compacto de $\mathbb{R}^n$ es Borel medible y por tanto puede ser aproximada por una red neuronal [18].

El Teorema de aproximación universal es un enunciado de **existencia**: garantiza que, para la función objetivo, existe un conjunto de pesos y sesgos θ^* con el que la red la aproxima con precisión arbitraria. No obstante, esto no garantiza que el algoritmo de entrenamiento sea capaz de aprenderla. Aunque el MLP pueda, en teoría, representarla, el proceso de aprendizaje puede fallar por dos motivos. Primero, el algoritmo de optimización utilizado para el entrenamiento podría no converger al conjunto de parámetros correspondiente a la función deseada. Segundo, el algoritmo de entrenamiento podría elegir la función incorrecta debido al sobreajuste.

Esta limitación se relaciona con el Teorema de «no hay almuerzo gratis» de la Sección 2.2.5, según el cual ningún algoritmo de aprendizaje es universalmente mejor que los demás. Las redes prealimentadas garantizan que cualquier función puede representarse, pero el teorema implica que ninguna arquitectura ni procedimiento de entrenamiento es óptimo para todos los problemas. Por eso la elección del modelo y del método de entrenamiento debe apoyarse en las características de cada problema, y no en la búsqueda de un método universalmente superior [8].

3.5. Regularización

Como se ha discutido previamente, la generalización es uno de los problemas centrales del aprendizaje automático. La **regularización** engloba las técnicas destinadas a reducir el error de generalización, aunque pueda implicar un aumento del error sobre el conjunto de entrenamiento. Estas estrategias limitan, de forma explícita o implícita, la capacidad del modelo para ajustarse a fluctuaciones particulares de la muestra. Algunos enfoques habituales son [8]:

- **Penalización de los parámetros.** Modifica la función objetivo añadiendo términos que restringen la magnitud o estructura de los pesos del modelo. Los ejemplos más comunes son la regularización L^2 y L^1 .
- **Regularización mediante datos.** Aumenta la variabilidad efectiva del conjunto de entrenamiento o introduce invariancias relevantes para el problema. El ejemplo principal es el *data augmentation*.
- **Regularización durante el entrenamiento.** Controla la dinámica de optimización para evitar que el modelo continúe ajustándose al ruido. La parada temprana, o *early*

stopping, pertenece a esta categoría.

- **Regularización estocástica de la arquitectura.** Introduce aleatoriedad en la red durante el entrenamiento para reducir la coadaptación entre unidades. El caso más habitual es el *dropout*.

3.5.1. Penalización a la norma de los parámetros

La penalización a la norma de los parámetros es un enfoque clásico de regularización que añade un término de penalización en norma, denotado $\Omega(\boldsymbol{\theta})$, a la función objetivo J . Denotamos a la función objetivo regularizada $\tilde{J}$:

$$\tilde{J}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) + \alpha\Omega(\boldsymbol{\theta}), \quad (3.10)$$

donde $\alpha \in [0, \infty)$ es un hiperparámetro que controla el peso relativo del término de penalización en norma relativo a la función objetivo estándar J .

El entrenamiento del algoritmo da lugar a una minimización de la función original J y de la norma de los parámetros $\boldsymbol{\theta}$. Diferentes elecciones de la norma Ω resultan en diferentes soluciones alcanzadas, siendo las más destacadas la norma $\|\cdot\|_2$ y la norma $\|\cdot\|_1$.

En redes neuronales es habitual escoger una norma que penalice sólo a los pesos y dejar a los sesgos sin regularizar. Los sesgos normalmente se ajustan a los datos con menos datos que los pesos. Regularizar los sesgos puede introducir un nivel significativo de subajuste [8].

Regularización L^2

Es una de las estrategias más simples y comunes, también conocida como **regresión Ridge** o **regularización de Tikhonov**. El término de penalización viene dado por:

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{\theta}\|_2^2, \quad (3.11)$$

que fuerza los pesos del modelo a acercarse al origen. Para entenderlo, consideremos un ejemplo de entrenamiento de una red neuronal en el que no hay término de penalización en el sesgo:

$$\tilde{J}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = \frac{\alpha}{2}\boldsymbol{\theta}^\top \boldsymbol{\theta} + J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}), \quad (3.12)$$

con el correspondiente gradiente:

$$\nabla_{\boldsymbol{\theta}} \tilde{J}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = \alpha\boldsymbol{\theta} + \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}). \quad (3.13)$$

Aplicando el proceso de aprendizaje descrito en secciones anteriores, actualizamos los pesos restándoles el gradiente multiplicado por una tasa de aprendizaje ϵ :

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon(\alpha\boldsymbol{\theta} + \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y})). \quad (3.14)$$

Reescribiendo, la actualización resulta en:

$$\boldsymbol{\theta} \leftarrow (1 - \epsilon\alpha)\boldsymbol{\theta} - \epsilon\nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) \quad (3.15)$$

Podemos observar que el término de penalización ha modificado la regla de aprendizaje para encoger el vector de parámetros multiplicándolo por una tasa constante: $(1 - \epsilon\alpha)$. Por este motivo, esta regularización también se conoce como *weight decay* (decaimiento de los pesos) [8].

Regularización L^1

Aunque la regularización L^2 es la más común, la regularización L^1 o **Lasso** ofrece una forma alternativa de penalizar el tamaño de los parámetros del modelo. Formalmente, el término de penalización se define como:

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_i |\theta_i|. \quad (3.16)$$

A continuación discutiremos el efecto de la regularización L^1 del mismo modo que lo hicimos en el caso anterior. Consideremos de nuevo una red neuronal sin término de penalización para el sesgo:

$$\tilde{J}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = \alpha\|\boldsymbol{\theta}\|_1 + J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}), \quad (3.17)$$

con el gradiente correspondiente:

$$\nabla_{\boldsymbol{\theta}}\tilde{J}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = \alpha\text{signo}(\boldsymbol{\theta}) + \nabla_{\boldsymbol{\theta}}J(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}), \quad (3.18)$$

donde $\text{signo}(\boldsymbol{\theta})$ es simplemente el signo de $\boldsymbol{\theta}$ aplicado elemento por elemento. Al inspeccionar la Ecuación 3.18 podemos observar que el efecto de la regularización L^1 es distinto al de la regularización L^2 . Específicamente, podemos ver que la contribución de la regularización al gradiente ya no escala linealmente con cada parámetro θ_i ; en su lugar, es un factor constante con un signo igual a $\text{signo}(\theta_i)$. Una de las implicaciones de esto es que las expresiones algebraicas para obtener simplificaciones de J se vuelven más complicadas [8].

3.5.2. *Data augmentation*

El aumento de datos o *data augmentation* mejora la generalización de los modelos de aprendizaje automático cuando los datos de entrenamiento son limitados, mediante la generación de datos sintéticos.

Este enfoque es directo en problemas de clasificación: dado un par entrada-etiqueta $(\mathbf{x}, y)$, se generan nuevos datos aplicando transformaciones sobre $\mathbf{x}$ que mantengan invariante la categoría y . También es altamente eficaz en visión artificial. Operaciones geométricas como traslaciones, rotaciones o escalados fuerzan al modelo a aprender características invariantes espacialmente (ver Figura 3.5). La restricción es que la transformación no debe alterar la

clase del objeto: voltear horizontalmente una «d» la transforma en una «b», alterando su significado por completo [8].

Añadir pequeños niveles de **ruido aleatorio** actúa como una forma de aumento de datos. Dado que las redes neuronales carecen de robustez frente al ruido, inyectarlo durante el entrenamiento fuerza al modelo a volverse robusto [8].



Figura 3.5 *Dataset augmentation*. Ejemplo de transformaciones que no alteran el significado de la imagen (Facultad de Ciencias). Todas preservan la categoría del objeto, generando nuevas muestras de entrenamiento sin alterar su etiqueta.

3.5.3. Parada temprana

A medida que una red neuronal se entrena durante múltiples iteraciones o épocas, el error de la función de coste evaluada sobre el conjunto de entrenamiento tiende a disminuir monótonamente. Sin embargo, el error evaluado sobre un conjunto de validación independiente típicamente desciende hasta alcanzar un mínimo, tras el cual comienza a aumentar (ver Figura 3.6). Este punto de inflexión marca el inicio del sobreajuste [8].

La **parada temprana** o *early stopping* es una forma de regularización en el tiempo que explota directamente este comportamiento. El algoritmo monitoriza una métrica objetivo en el conjunto de validación al final de cada época. El entrenamiento se interrumpe automáticamente cuando dicha métrica deja de mejorar durante un número predefinido de épocas consecutivas, parámetro conocido como **paciencia**. Tras la interrupción, el algoritmo descarta los pesos del modelo actual y restaura los parámetros correspondientes a la época que produjo el mejor rendimiento histórico en validación. Este procedimiento se detalla con mayor

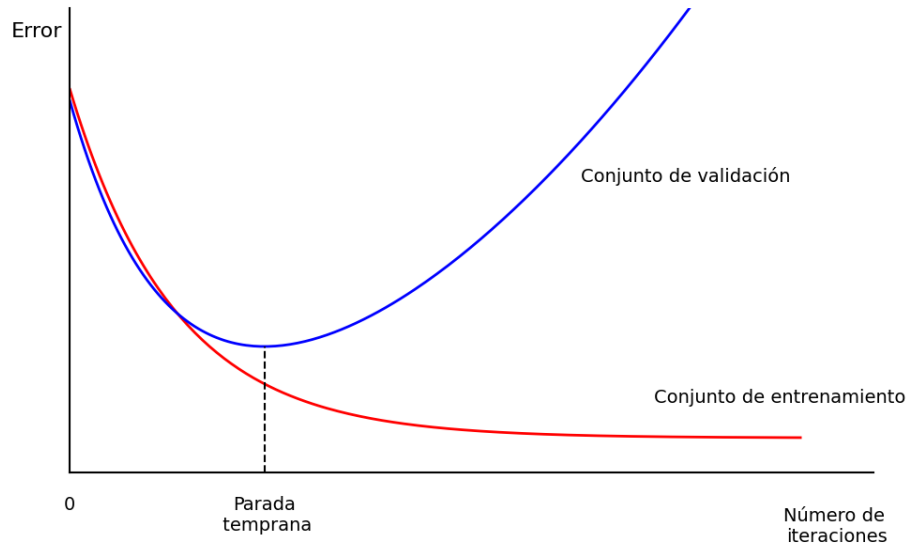


Figura 3.6 Representación de las curvas de aprendizaje respecto al número de iteraciones (épocas). La parada temprana minimiza el error en el conjunto de validación para evitar el sobreajuste.

profundidad en el Algoritmo 5. Es una estrategia eficiente que elimina la necesidad de ajustar el número total de épocas como hiperparámetro.

Algoritmo 5: Parada Temprana

Entrada: n (pasos), p (paciencia)

Salida: θ^*, i^*

```

1  $\theta, \theta^* \leftarrow \theta_0; i, j, i^* \leftarrow 0; v \leftarrow \infty$ 
2 mientras  $j < p$  hacer
3   Actualizar  $\theta$  entrenando  $n$  pasos;  $i \leftarrow i + n; v' \leftarrow \text{ErrorVal}(\theta)$ 
4   si  $v' < v$  entonces
5      $j \leftarrow 0; v \leftarrow v'; \theta^* \leftarrow \theta; i^* \leftarrow i$ 
6   en otro caso
7      $j \leftarrow j + 1$ 
8   fin
9 fin
```

3.5.4. Dropout

El **dropout** es un método de regularización barato computacionalmente y muy efectivo. Durante la fase de entrenamiento, el dropout establece a cero aleatoriamente una fracción de las neuronas de una capa en cada iteración, junto con sus conexiones correspondientes (ver Figura 3.7). La probabilidad de que una neurona sea retenida se define por un hiperparámetro $p \in (0, 1]$. Esto es equivalente a multiplicar el vector de activaciones por una máscara binaria cuyas entradas siguen una distribución de Bernoulli de parámetro p [8].

El dropout rompe la **dependencia mutua** entre neuronas. Al desactivarlas al azar, evitamos que una neurona solo aprenda a corregir los errores de sus compañeras, obligando a cada una a extraer información útil por sí misma. Durante la fase de inferencia, todas las neuronas permanecen activas, pero los pesos de salida se escalan por el factor p para mantener el valor esperado de las activaciones. Desde una perspectiva teórica, esta técnica puede interpretarse como un método que entrena y promedia un gran ensamblaje de subredes que comparten pesos [8].

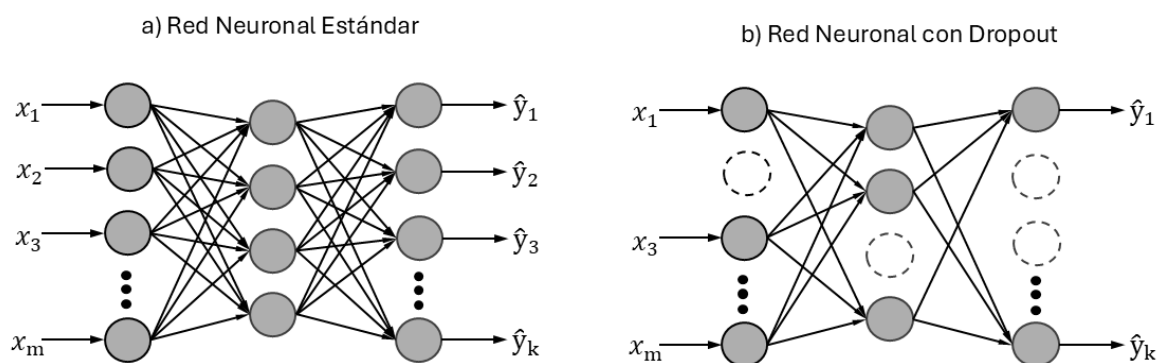


Figura 3.7 Representación del funcionamiento del *dropout*. El ensamblaje está formado por los modelos en los que se eliminan neuronas no pertenecientes a la capa de salida. La imagen representa uno de esos modelos. Imagen adaptada de [19].

3.6. Transformadores

En esta sección introducimos los **transformadores**, la arquitectura estándar para la construcción de **grandes modelos de lenguaje** (LLM, *Large Language Models*). Esta arquitectura fue propuesta originalmente para tareas de traducción automática en el procesamiento de lenguaje natural (NLP, *Natural Language Processing*) en 2017, en el artículo de Vaswani et al. [20] *Attention is All You Need*. Fue desarrollada con el fin de superar las limitaciones que las redes neuronales recurrentes, como las unidades LSTM (*Long Short Term Memory*) y GRU (*Gated Recurrent Unit*) presentaban a la hora de procesar secuencias largas y paralelizar el entrenamiento [21].

El elemento diferenciador de los transformadores es que prescinden por completo de los mecanismos recurrentes, introduciendo en su lugar el algoritmo de **autoatención** o *self-attention*. Este mecanismo permite modelar el contexto y las dependencias temporales entre las palabras de forma global y simultánea [21].

El éxito de esta arquitectura en el procesamiento de texto impulsó su extensión a otros ámbitos, como los **Vision Transformers** (ViT) en visión artificial [22]. Los ViT aplican la arquitectura del transformador directamente sobre secuencias de parches de imagen, omitien-

do las capas convolucionales. A gran escala, alcanzan resultados competitivos en numerosas tareas de visión. [10].

3.6.1. Mecanismo de la atención

El mecanismo de atención permite construir representaciones dependientes del contexto. Una misma palabra puede adquirir significados distintos según los términos que la acompañan. Por ejemplo, «banco» no tiene la misma interpretación en «banco de peces» que en «banco para sentarse». La autoatención formaliza esta idea: cada elemento de la secuencia combina información del resto, ponderando más los que resultan más relevantes.

Antes de introducir este mecanismo, es necesario describir cómo una secuencia textual se transforma en una entrada numérica para el modelo. Este proceso consta de dos etapas: la **tokenización**, que divide el texto en unidades discretas, y la construcción de **embeddings**, que asocia a cada token un vector en un espacio de representación continuo.

Tokenización

El primer paso del procesamiento textual es la **tokenización**. Mediante este proceso, el texto original se transforma en unidades más pequeñas o **tokens** pertenecientes a un **vocabulario**.

En una primera aproximación resulta natural considerar a cada palabra como un token. No obstante, habría palabras que se quedarían fuera, como los nombres propios, por ejemplo. Por otro lado, están los signos de puntuación: un signo de interrogación afecta enormemente al significado de una oración. También están los morfemas, un diccionario que considere a las palabras «estar, estando, estás» como diferentes es ineficiente computacionalmente [21].

En la práctica, se utiliza un compromiso entre palabras, letras y signos de puntuación. El vocabulario final contiene palabras comunes y fragmentos de palabras a partir de los cuales construir palabras menos comunes. Una vez completada la tokenización, a cada palabra se le asigna un identificador (ID) en base al diccionario.

Embeddings

Cada token en el vocabulario $\mathcal{V}$ se asocia a un único **embedding**. Un embedding es un vector de dimensión d que representa el token de entrada. El diccionario donde se almacenan estos embeddings es la **matriz de embeddings** $\mathbf{E}$. Dado que esta matriz tiene una columna por cada palabra en el vocabulario, sus dimensiones son $\mathbb{R}^{d \times |\mathcal{V}|}$.

De este modo, los N tokens de entrada primero se transforman en una representación $\mathbf{T} \in \mathbb{R}^{|\mathcal{V}| \times N}$ donde la n -ésima columna es un vector one-hot correspondiente al n -ésimo token de la secuencia. La entrada al transformador finalmente se calcula como $\mathbf{X} = \mathbf{ET}$, y los

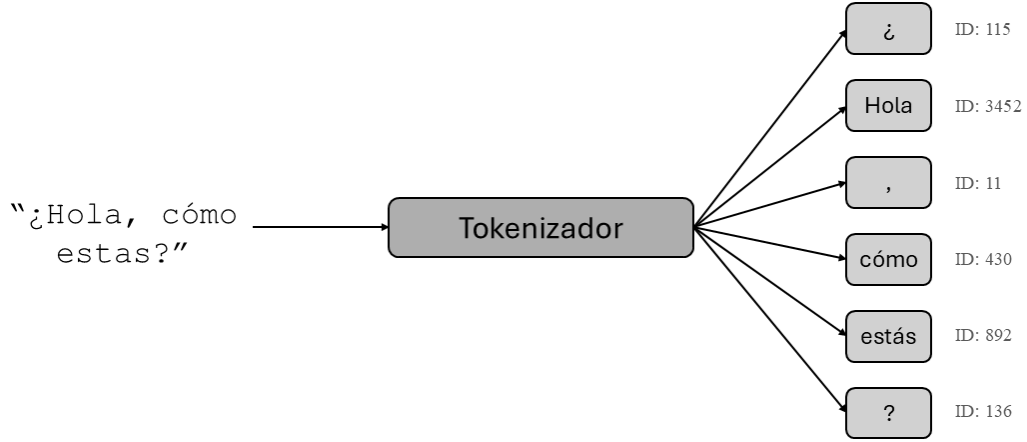


Figura 3.8 Ejemplo visual del proceso de tokenización. La frase de entrada se divide en subunidades, signos de puntuación incluidos y a cada token se le asigna un identificador numérico único basado en el vocabulario del modelo. Los identificadores son representativos, no muestran valores pertenecientes a ningún diccionario real.

parámetros de la matriz $\mathbf{E}$ se aprenden durante el entrenamiento. Un valor habitual para d es 1.024, mientras que para $|\mathcal{V}|$ es de 30.000 [10].

Los transformadores tienen además un mecanismo propio para inyectar información sobre el índice de posición del token en la secuencia, conocido como **codificación posicional**, que simplemente se suma al embedding original.

Para inyectar el orden de los tokens, el modelo original de Vaswani et al. propone sumar un vector de posición al vector de embedding de la palabra. Asumiendo que la dimensión del embedding d es par, la matriz de codificación posicional $\mathbf{P} \in \mathbb{R}^{N \times d}$ se define mediante funciones sinusoidales de diferentes frecuencias, de modo que la fila j -ésima de $\mathbf{P}$ es el vector que se suma al embedding del j -ésimo token. Para $j = 0, 1, \dots, N - 1$ e $i = 0, 1, \dots, \frac{d}{2} - 1$, las componentes par e impar se calculan como:

$$PE_{(j, 2i)} = \sin\left(\frac{j}{10000^{2i/d}}\right), \quad (3.19)$$

$$PE_{(j, 2i+1)} = \cos\left(\frac{j}{10000^{2i/d}}\right). \quad (3.20)$$

Cada par de dimensiones $(2i, 2i + 1)$ codifica una senoide con frecuencia $\omega_i = (1/10000)^{2i/d}$, que decrece exponencialmente con i : las primeras dimensiones capturan posiciones locales con frecuencias altas, mientras que las últimas codifican dependencias globales con frecuencias bajas [20].

Autoatención

Como hemos descrito anteriormente, la atención es una operación que permite calcular una representación vectorial de un token integrando la información de su contexto. La atención toma la representación $\mathbf{x}_i$ de un token en la posición i , evalúa una ventana de tokens previos $\mathbf{x}_1, \dots, \mathbf{x}_{i-1}$, y produce un nuevo vector de salida $\mathbf{a}_i$.

De forma intuitiva, una capa de autoatención mapea una secuencia de entrada $(\mathbf{x}_1, \dots, \mathbf{x}_N)$ a una secuencia de salida $(\mathbf{a}_1, \dots, \mathbf{a}_N)$. Esta transformación puede entenderse como una suma ponderada de todos los tokens que lo preceden [21]:

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j. \quad (3.21)$$

Una única cabeza de atención

Para dotar a esta suma ponderada de mayor flexibilidad, los transformadores introducen el concepto de **cabeza de atención**. Este mecanismo se inspira en los sistemas de recuperación de información y asigna tres roles distintos a cada vector de entrada durante el proceso:

- **Consulta** (*Query*, $\mathbf{q}_i$): Representa la pregunta, es decir, «¿qué información está buscando el token actual?».
- **Clave** (*Key*, $\mathbf{k}_j$): Funciona como un identificador, respondiendo a «¿qué información contiene este token previo?».
- **Valor** (*Value*, $\mathbf{v}_j$): Representa el contenido semántico real que el token aportará a la suma final si su clave coincide con la consulta.

Para capturar estos roles, la red aprende tres matrices de pesos $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$, que proyectan el vector de entrada original $\mathbf{x}_i$ en sus respectivos espacios de representación:

$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i; \quad \mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i; \quad \mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i. \quad (3.22)$$

La relevancia del token j para el token i se mide mediante el producto escalar entre la consulta $\mathbf{q}_i$ y la clave $\mathbf{k}_j$. Para evitar que estos productos crezcan excesivamente con la dimensión, se aplica el escalado por $\sqrt{d_k}$:

$$s_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}. \quad (3.23)$$

A continuación, las puntuaciones s_{ij} se transforman en pesos de atención mediante la función softmax:

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell \leq i} \exp(s_{i\ell})}. \quad (3.24)$$

Así, los coeficientes α_{ij} son no negativos y satisfacen $\sum_{j \leq i} \alpha_{ij} = 1$, por lo que pueden interpretarse como una distribución de pesos sobre los tokens anteriores. La salida de la cabeza

de atención se obtiene como una combinación convexa de los vectores de valor:

$$\mathbf{head}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j. \quad (3.25)$$

Finalmente, se aplica una proyección lineal de salida¹[21]:

$$\mathbf{a}_i = \mathbf{W}^O \mathbf{head}_i. \quad (3.26)$$

Atención multicabeza Hasta ahora hemos descrito una única cabeza de atención. Sin embargo, en la práctica, los transformadores utilizan múltiples cabezas. La intuición detrás de este diseño es que cada cabeza puede atender al contexto con propósitos diferentes: algunas cabezas podrían especializarse en relaciones sintácticas o semánticas, mientras que otras podrían buscar patrones secuenciales [21].

Por lo tanto, en la **atención multicabeza** tenemos A cabezas diferentes que operan en paralelo dentro de la misma capa del modelo. Cada cabeza c tiene sus propias matrices de consulta, clave y valor: $\mathbf{W}^{Q_c}$, $\mathbf{W}^{K_c}$ y $\mathbf{W}^{V_c}$, de modo que cada una es capaz de modelar diferentes aspectos semánticos.

La dimensión del modelo d se mantiene para la entrada y la salida global. Los vectores de consulta y clave tienen una dimensionalidad d_k , y los vectores de valor tienen una dimensionalidad d_v . Así, asumiendo vectores columna de entrada $\mathbf{x}_i \in \mathbb{R}^{d \times 1}$, para cada cabeza c se tienen matrices de pesos $\mathbf{W}^{Q_c} \in \mathbb{R}^{d_k \times d}$, $\mathbf{W}^{K_c} \in \mathbb{R}^{d_k \times d}$, y $\mathbf{W}^{V_c} \in \mathbb{R}^{d_v \times d}$.

3.6.2. Bloques del transformador

El cálculo de la autoatención es el núcleo de lo que llamamos un **bloque del transformador**. Este incluye, además de la capa de la autoatención, otros tres tipos de capas: (1) una capa prealimentada, (2) conexiones residuales y (3) capas de normalización.

La Figura 3.9 representa el funcionamiento de un bloque del transformador. Primero, la entrada atraviesa una capa de atención multicabeza y es añadida a la salida de esta mediante una conexión residual. Después se introduce a una red MLP completamente conectada. Además, se añade una capa de normalización a la salida de cada una.

La capa de normalización utiliza una variación del z -score estadístico, diseñada para mantener los valores de las activaciones en un rango que facilite el entrenamiento basado en gradientes. A diferencia de otras normalizaciones, esta se aplica al vector de un único token.

Para un vector $\mathbf{x}$ de dimensión d , se calculan la media μ y la desviación típica σ de sus

¹Una demostración visual e intuitiva sobre la interacción de las matrices Consulta, Clave y Valor en el algoritmo de atención puede consultarse en el siguiente vídeo: <https://www.youtube.com/watch?v=RNFOFvRjGZk>.

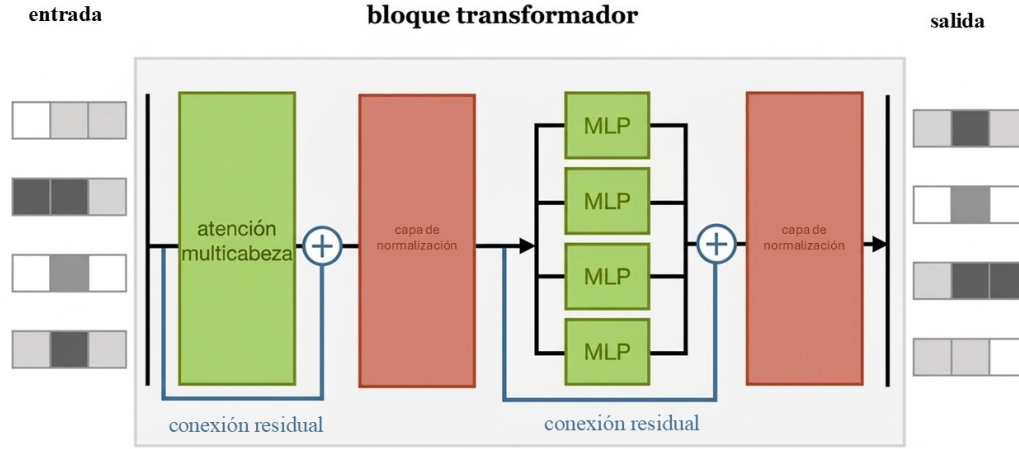


Figura 3.9 Arquitectura de un bloque transformador. La entrada es una matriz de $D \times N$, donde cada columna representa un token. El bloque procesa la información mediante atención multicabeza y redes MLP, utilizando conexiones residuales y normalización de capa para preservar el flujo de gradientes. Imagen adaptada de [23].

componentes:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}. \quad (3.27)$$

Posteriormente, el vector se normaliza sustrayendo la media y dividiendo por la desviación típica. El resultado es un vector de media cero y desviación típica 1:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sigma}. \quad (3.28)$$

Finalmente, la implementación estándar de la normalización de capa incluye dos parámetros de aprendizaje γ (ganancia) y β (desplazamiento):

$$\text{NormalizacionCapa}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sigma} + \beta, \quad (3.29)$$

donde $\odot$ representa el **producto de Hadamard** o elemento a elemento.

Observamos que las dimensiones de entrada y de salida de un bloque coinciden, de modo que estos pueden ser apilados. Un transformador completo es una colección L bloques a través de los cuales fluye la información gracias a las conexiones residuales. Modelos como GPT-3-small apilan 12 de estas capas, mientras que modelos más grandes como GPT-3-large apilan hasta 96 [21].

3.6.3. La capa de salida

El último componente de un transformador que debemos introducir es la **cabeza de modelado de lenguaje**. Este módulo es el encargado de transformar la representación abstracta final del modelo en una predicción tangible sobre el vocabulario.

El objetivo de un modelo de lenguaje es actuar como un predictor de palabras. Dada una secuencia de tokens, debe asignar una probabilidad a cada palabra posible del vocabulario $\mathcal{V}$ para la siguiente posición. Mientras que los modelos clásicos de n -gramas limitaban su contexto a $n-1$ palabras, los transformadores modernos pueden manejar ventanas de contexto de miles o incluso millones de tokens [21].

Específicamente, esta capa toma la salida del último bloque del transformador correspondiente al último token de la secuencia, denotada como el vector columna $\mathbf{h}_N^L \in \mathbb{R}^{d \times 1}$. El proceso consta de dos pasos fundamentales:

1. Capa de **des-embedding**. Se realiza una proyección lineal para mapear el vector del espacio de características al espacio del vocabulario. Aunque se podría utilizar una matriz de pesos independiente, lo más habitual es emplear la técnica de atadura de pesos o *weight tying*, reutilizando la traspuesta de la matriz de embeddings original $\mathbf{E} \in \mathbb{R}^{d \times |\mathcal{V}|}$ [21]. Así, el vector de **logits** se calcula como:

$$\mathbf{u} = \mathbf{E}^\top \mathbf{h}_N^L \in \mathbb{R}^{|\mathcal{V}| \times 1}. \quad (3.30)$$

En este proceso, $\mathbf{E}$ se optimiza para ser efectiva tanto en la proyección inicial como en la inversa.

2. Generación de probabilidades. La función softmax se aplica sobre el vector $\mathbf{u}$ para obtener el vector de probabilidades:

$$\mathbf{y} = \text{softmax}(\mathbf{u}) \in \mathbb{R}^{|\mathcal{V}| \times 1} \quad (3.31)$$

Finalmente, a partir de esta distribución $\mathbf{y}$, el modelo genera texto seleccionando un token mediante diversos métodos de muestreo, como la **búsqueda codiciosa**, donde se elige el índice k con la probabilidad máxima.

Los transformadores que utilizan esta configuración suelen denominarse modelos **solo decodificador**. Aunque el transformador original fue diseñado como una arquitectura codificador-decodificador orientada a la traducción, la mayor parte de los modelos de lenguaje actuales (como las familias GPT, Llama o Claude) emplean esta versión simplificada y eficiente para la generación de texto [21].

3.6.4. Variantes de la arquitectura

La arquitectura descrita en las secciones anteriores corresponde a un bloque transformador basado en autoatención. Dependiendo de cómo se aplique la atención y de la tarea considerada, estos bloques pueden organizarse en tres configuraciones principales: codificadores, decodificadores y arquitecturas codificador-decodificador.

- **Codificador.** Procesa toda la secuencia de entrada de forma simultánea, de modo que cada token puede atender al resto de elementos de la secuencia. Su objetivo no es generar una secuencia nueva, sino construir representaciones vectoriales de la entrada. Esta configuración es especialmente relevante en modelos como BERT y en arquitecturas de visión basadas en transformadores, como ViT.
- **Decodificador.** Restringe la atención únicamente a los tokens anteriores. Esto permite generar texto de forma autorregresiva, prediciendo cada token a partir del contexto previo. Es la arquitectura característica de modelos de lenguaje generativos como GPT.
- **Codificador-decodificador.** Combina un codificador que representa la secuencia de entrada con un decodificador que genera la secuencia de salida. Esta fue la arquitectura original propuesta por Vaswani et al. y resulta natural en tareas de traducción automática, donde entrada y salida son secuencias distintas.

En el contexto de este trabajo, la configuración más relevante es la de codificador. CLIP no se utiliza como modelo generativo, sino como un sistema de representación multimodal: un codificador de imágenes y un codificador de texto proyectan fotografías y descripciones textuales a un espacio común, donde pueden compararse mediante medidas de similitud. Por ello, el interés principal recae en los transformadores como extractores de representaciones, especialmente en su variante visual, los *Vision Transformers*.

4. Visión artificial

La **visión artificial** es la disciplina de la inteligencia artificial centrada en extraer información semántica y estructural a partir de imágenes digitales. Durante décadas, el enfoque predominante consistía en el diseño manual de características. Los ingenieros especificaban explícitamente qué patrones buscar en una imagen como bordes, texturas o gradientes. Este enfoque presentaba un límite de rendimiento claro pues la variabilidad visual del mundo real no es modelable mediante reglas estáticas [24].

A partir de 2012 [25], el aprendizaje profundo cambió el campo, desplazando el diseño manual de características por representaciones aprendidas directamente de los datos. Este capítulo recorre la evolución de ese cambio de paradigma: desde el problema de reconocimiento de imágenes que lo inició [26], pasando por las arquitecturas convolucionales que lo hicieron posible [24, 25] y los Vision Transformers que lo llevaron más lejos [22], hasta llegar a CLIP, el modelo multimodal utilizado en este trabajo [6].

4.1. ImageNet y el problema del reconocimiento visual

El reconocimiento de imágenes es, en su enfoque más directo, un problema de clasificación. La dificultad no reside en la formulación sino en la escala y variabilidad del problema, pues una misma categoría (un gato, una silla, una ventana) puede aparecer en miles de contextos distintos con variaciones de iluminación, ángulo o posición.

Para poder medir el progreso de forma objetiva, Deng et al. [26] construyeron ImageNet, una base de datos de más de 14 millones de imágenes etiquetadas manualmente en más de 20.000 categorías organizadas según la jerarquía semántica de WordNet. A partir de 2010, el *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC) propuso una competición anual sobre un subconjunto de 1.000 categorías y 1,2 millones de imágenes de entrenamiento, convirtiéndose en el *benchmark* de referencia para la visión artificial durante más de una década.

Durante los primeros años de la competición, los mejores sistemas estaban basados en descriptores clásicos combinados con máquinas de vectores soporte, obteniendo tasas de error top-5 en torno al 25–26 %. En 2012 Krizhevsky, Sutskever e Hinton [25] presentaron AlexNet, una red neuronal convolucional profunda que redujo ese error al 15,3 %, una mejora que marcó el inicio del aprendizaje profundo en visión. En los años siguientes, arquitecturas sucesivas como VGGNet, GoogLeNet o ResNet continuaron reduciendo el error hasta niveles que superan la precisión humana en esa tarea concreta.

4.2. Redes neuronales convolucionales

Las redes neuronales convolucionales (CNN, *Convolutional Neural Networks*) [24] son la arquitectura que hizo posible el salto de rendimiento descrito en la sección anterior. Su principio fundamental es la convolución discreta. En lugar de conectar cada neurona con todos los píxeles de la imagen –lo que produciría un número de parámetros inmanejable–, cada neurona se conecta únicamente con una región local de la entrada, compartiendo los mismos pesos a lo largo de toda la imagen.

Las capas convolucionales son el elemento fundamental de las CNN. Consisten en una serie de filtros, llamados **kernels**, que se aplican de manera deslizante sobre el tensor de entrada. Estos kernels están definidos por una matriz de pesos que se optimizan durante el entrenamiento. Dada una imagen bidimensional I como entrada, y un kernel K , la convolución se define como¹:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n), \quad (4.1)$$

donde a S se le denomina **mapa de características**. Cada filtro detecta patrones locales como bordes, curvas o texturas. Al apilar capas, la red construye representaciones jerárquicas: las primeras capas detectan patrones simples; las intermedias, combinaciones; las finales, conceptos semánticos [8].

Las CNN incorporan además capas de *pooling* que reducen la resolución espacial y confieren invarianza a pequeñas traslaciones, y capas completamente conectadas al final de la red, que realizan la clasificación (ver Figura 4.1). Dos propiedades clave las hacen especialmente adecuadas para imágenes: la **localidad**, que reduce el número de parámetros, y la **equivalencia a traslaciones**, por lo que el mismo patrón se detecta independientemente de dónde aparezca [10].

4.3. Transformadores para imágenes

Los transformadores fueron originalmente diseñados para el procesamiento de texto. Su éxito en este ámbito llevó a la experimentación con imágenes. En su adaptación a la visión, los transformadores se enfrentaban a dos limitaciones principales frente a las CNN:

1. Hay muchos más píxeles en una imagen que palabras en una oración. Si N es la longitud de la entrada y D el número de dimensiones de cada elemento de esta, la autoatención tiene una complejidad computacional de $\mathcal{O}(N^2D)$. Este orden cuadrático supone un cuello de botella si se atiende a cada píxel individualmente.

¹En el contexto del aprendizaje profundo se utiliza el término convolución por convención, aunque matemáticamente la operación se define como correlación cruzada. La convolución teórica requiere invertir el kernel.

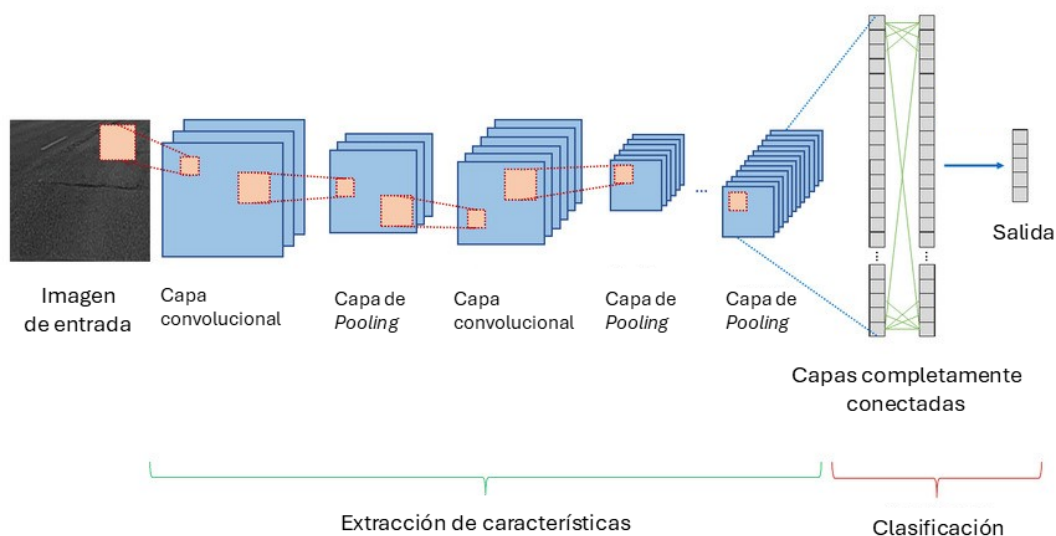


Figura 4.1 Estructura de una red neuronal convolucional. La red está formada por una sucesión de filtros, llamados kernels, y capas de *pooling*. La parte final de la red está formada por una red completamente conectada que realiza la clasificación. Imagen adaptada de [27].

2. Las redes convolucionales tienen un excelente sesgo inductivo, pues cada capa es equi-variante ante la traslación espacial y aprovechan inherentemente la estructura 2D de la imagen. Por el contrario, en un transformador puro, esta estructura espacial debe ser aprendida desde cero por la red.

A pesar de estas limitaciones, los transformadores para imagen han superado a las redes convolucionales en clasificación y otras tareas [28], principalmente por la escala a la que pueden construirse y la cantidad de datos que aprovechan durante el entrenamiento.

Para resolver el problema del coste cuadrático, el trabajo de Dosovitskiy et al. [22] introdujo la idea de dividir la imagen en parches de 16×16 píxeles. Cada parche se aplanar y se proyecta mediante una transformación lineal aprendida. Posteriormente, estos resultados se combinan con un vector de codificación posicional antes de introducirse al transformador (ver Figura 4.2).

El ViT solo logró superar a su *baseline* convolucional BiT [29] cuando utilizó más de 100 millones de imágenes de entrenamiento del conjunto JFT-300M. Al usar ImageNet por sí solo, o un subconjunto aleatorio de 10 o 30 millones de muestras de JFT-300M, el modelo ViT por lo general tuvo un desempeño muy inferior al de BiT. Estos resultados sugieren que, en ámbitos donde los conjuntos de datos son reducidos, los sesgos inductivos presentes en las convoluciones resultan extremadamente valiosos. No obstante, cuando se cuenta con cantidades de datos varios órdenes de magnitud mayor, el modelo de tipo transformador logra descubrir representaciones aún superiores, que van más allá de lo que puede alcanzar una red neuronal convolucional tradicional [22].

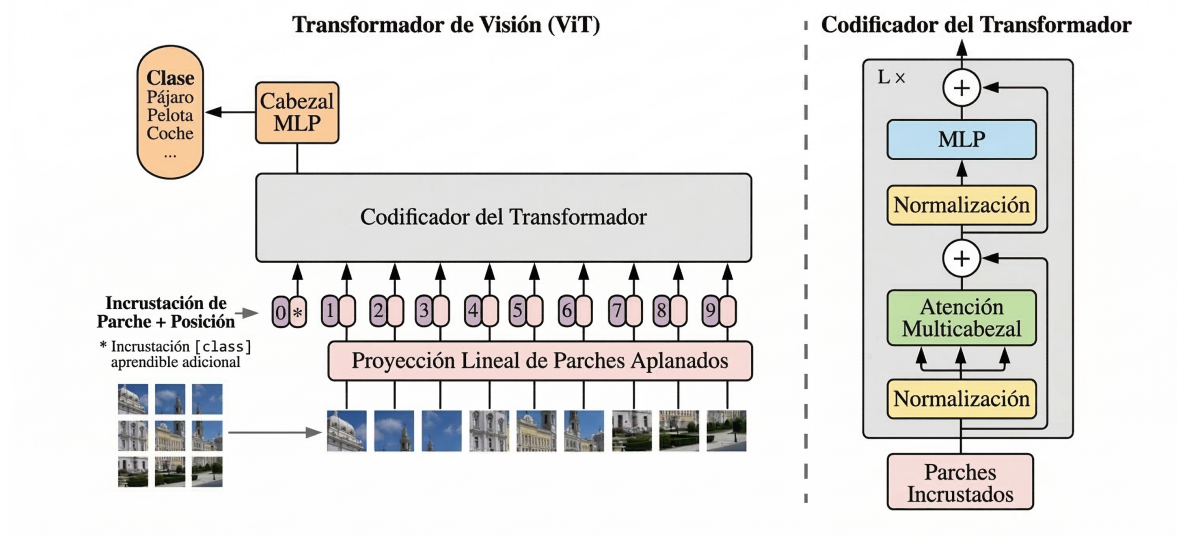


Figura 4.2 El modelo de transformador de visión ViT introducido por Dosovitskiy et al. La imagen se divide en parches de 16×16 píxeles. Cada parche es aplanado y pasado a través de una matriz de proyección lineal. Estas entradas se procesan a través de un codificador Transformer (derecha) varias veces antes de predecir la clase de la imagen. Imagen traducida y adaptada de [30].

4.4. El modelo CLIP

Los sistemas de visión artificial tradicionales se entrenaban para predecir un conjunto fijo de categorías de objetos predeterminadas. Esta restricción limitaba su generalidad, exigiendo nuevos datos etiquetados para cada concepto visual. Para superar esta limitación, Radford et al. [6] introdujeron CLIP (*Contrastive Language-Image Pre-training*), un modelo que aprende representaciones visuales directamente a partir de texto asociado a imágenes, aprovechando una fuente de supervisión masiva.

CLIP se entrena de forma contrastiva sobre 400 millones de pares (imagen, texto) extraídos de internet. Gracias a este enfoque, admite transferencia *zero-shot*: resuelve tareas de clasificación en conjuntos de datos nuevos sin reentrenamiento ni ajuste específico. En la práctica, CLIP actúa como un clasificador dinámico que evalúa imágenes a partir de descripciones en lenguaje natural, como «una foto de un perro».

A lo largo de este capítulo, se desglosará el funcionamiento interno de CLIP, comenzando por la arquitectura de sus codificadores y su función de pérdida contrastiva. Finalmente, se detallará la metodología empleada para extraer las puntuaciones numéricas de las imágenes que posteriormente se fusionarán con las variables tabulares clásicas en el modelo de predicción de precios.

4.4.1. Entrenamiento del modelo

Construcción de la base de datos

Para la construcción del modelo CLIP, Radford et al. [6] construyeron una base de datos con 400 millones de pares (imagen,texto) de una colección de fuentes públicas disponibles en internet. Para cubrir la mayor cantidad de conceptos visuales posible, buscaron parejas (imagen,texto) cuyo texto incluía una consulta entre 500.000. Este catálogo se construyó con todas las palabras que aparecían al menos 100 veces en la versión en inglés de la Wikipedia. Los resultados se balancearon para tener como máximo 20.000 pares (imagen,texto) por consulta.

Enfoque contrastivo

Los sistemas de visión artificial de última generación utilizan cantidades muy grandes de cálculo. En investigaciones previas, modelos como ResNeXt101-32x48d [31] requirieron 19 años de GPU para su entrenamiento, mientras que arquitecturas como Noisy Student EfficientNet L2 [32] demandaron 33 años-núcleo de TPuv3. Dado que estos sistemas fueron entrenados solo para 1.000 clases de ImageNet, aprender un conjunto abierto de conceptos visuales a partir de lenguaje natural parecía inviable computacionalmente.

En sus esfuerzos iniciales, los creadores de CLIP intentaron entrenar conjuntamente una red neuronal convolucional para imágenes y un transformador de texto desde cero para predecir la leyenda exacta de una imagen. Sin embargo, este enfoque resultaba ineficiente pues predecir las palabras exactas que acompañan a una imagen es una tarea extremadamente difícil. Esto se debe a la gran variedad de descripciones equivalentes que pueden existir para una fotografía dada. Por ello, decidieron cambiar este enfoque generativo a uno **contrastivo**, es decir, predecir únicamente qué texto completo se empareja con qué imagen, en lugar de predecir palabra por palabra. Se logró una mejora del 400 % en la eficiencia de la tasa de transferencia *zero-shot* [6].

Proceso de entrenamiento

El entrenamiento comienza tomando un subconjunto de N pares reales (imagen,texto). Para maximizar la eficiencia y la cantidad de ejemplos con los que el modelo aprende a contrastar, los autores fijaron $N = 32.768$.

A continuación, el lote de imágenes se pasa por un codificador de imágenes, que puede ser un modelo ResNet o un ViT, discutido en la Sección 4.3. Esto produce una matriz de características de imagen, que denotaremos $\mathbf{I}_f$. Simultáneamente, el modelo de texto pasa a través de un codificador de texto, similar a los detallados en la Sección 3.6, produciendo una matriz de características $\mathbf{T}_f$. En este punto, $\mathbf{I}_f$ y $\mathbf{T}_f$ pueden tener dimensiones diferentes.

Para poder comparar imágenes y texto, estos se llevan a un mismo espacio dimensional, el **espacio de incrustación multimodal**. Para ello, las representaciones extraídas $\mathbf{I}_f$ y $\mathbf{T}_f$, se multiplican por sus respectivas matrices de pesos aprendibles $\mathbf{W}_i$ y $\mathbf{W}_t$, y se normalizan mediante la norma L^2 , obteniendo los vectores normalizados $\mathbf{I}_e$ y $\mathbf{T}_e$:

$$\mathbf{I}_e = \frac{\mathbf{I}_f \mathbf{W}_i}{\|\mathbf{I}_f \mathbf{W}_i\|} \quad (4.2)$$

$$\mathbf{T}_e = \frac{\mathbf{T}_f \mathbf{W}_t}{\|\mathbf{T}_f \mathbf{W}_t\|}. \quad (4.3)$$

Una vez que ambas modalidades tienen la misma dimensión, el modelo calcula la similitud por pares. Al estar los vectores normalizados, la similitud coseno es equivalente a su producto escalar. Esta similitud se escala mediante un parámetro aprendible, llamado **temperatura** y denotado τ , para generar la matriz de puntuaciones o **logits** [6]:

$$\text{logits} = (\mathbf{I}_e \mathbf{T}_e^\top) \cdot e^\tau. \quad (4.4)$$

De las $N \times N$ combinaciones posibles en la matriz de logits, solo los N pares pertenecientes a la diagonal principal son los emparejamientos reales correctos. Los restantes $N^2 - N$ son emparejamientos incorrectos. CLIP optimiza una función de pérdida de entropía cruzada simétrica sobre estas puntuaciones. Esta función penaliza al modelo tanto si falla al buscar el texto correcto para una imagen dada (J_i) como si falla al buscar la imagen correcta para un texto dado (J_t). La pérdida total del lote se define como el promedio de ambas:

$$J = \frac{J_i + J_t}{2}. \quad (4.5)$$

4.4.2. Uso de CLIP para la predicción *zero-shot*

Una vez concluida la fase de entrenamiento, el modelo ha alineado los conceptos visuales y lingüísticos en un espacio común. El siguiente paso es utilizar esta representación para resolver problemas reales en conjuntos de datos no vistos previamente.

Clasificación

Para realizar la clasificación *zero-shot*, CLIP reutiliza la misma dinámica de emparejamiento contrastivo para la que fue preentrenado.

Dado un conjunto de K clases y una imagen objetivo, el procedimiento se divide en dos fases. En primer lugar, se construye el clasificador textual: para cada clase c_k se formula un *prompt* descriptivo p_k que se pasa por el codificador de texto, obteniendo un embedding normalizado $\mathbf{T}_e^{(k)}$. Dado que estas representaciones son independientes de la imagen, se calculan una sola vez y se almacenan en caché. En segundo lugar, la imagen objetivo se codifica y normaliza para

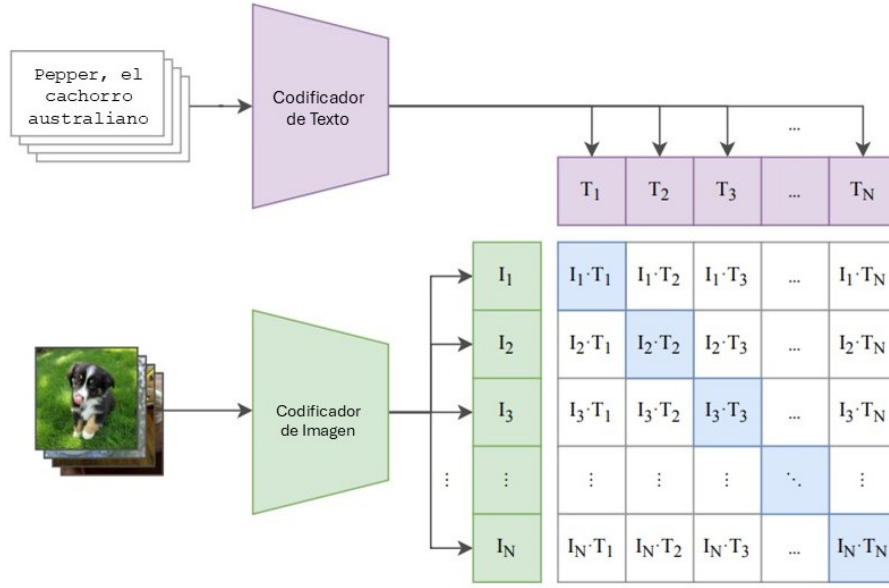


Figura 4.3 Entrenamiento del modelo CLIP. Dado un lote de N pares (imagen, texto), el modelo genera una representación de características $\mathbf{I}_f$ para la imagen y una $\mathbf{T}_f$ para el texto mediante los codificadores. Mediante proyecciones lineales y normalización L^2 se determinan los embeddings $\mathbf{I}_e$ y $\mathbf{T}_e$ finales. El entrenamiento maximiza la similitud coseno de los N pares correctos y minimiza la similitud de los $N^2 - N$ pares incorrectos. Imagen adaptada de [6].

obtener $\mathbf{I}_e$, y se calcula su similitud coseno frente a cada uno de los K embeddings textuales. Finalmente, estas puntuaciones se normalizan mediante una función softmax escalada por la temperatura τ , y el modelo predice la clase con mayor probabilidad resultante. El Algoritmo 6 resume este proceso.

Generación de atributos continuos

En el contexto de esta investigación, el objetivo no es realizar una clasificación categórica, sino cuantificar atributos visuales en una escala continua. Esta cuantificación nos permite generar nuevas variables numéricas que alimentarán nuestro modelo tabular de predicción de precios.

Para obtener estas métricas, se ha adaptado la mecánica de inferencia de CLIP mediante un enfoque de **pares semánticos dicotómicos**. Para cada atributo visual a que se desea medir, se definen dos *prompts* dicotómicos que representen valores opuestos. Por ejemplo, para evaluar la luminosidad de una estancia, definimos el par: $p_a^+ = \text{«Una foto de una habitación muy luminosa y llena de luz»}$ y $p_a^- = \text{«Una foto de una habitación muy oscura y sin luz»}$. Sus embeddings textuales normalizados son:

$$\mathbf{T}_e^{a,\pm} = \frac{\mathbf{T}_f(p_a^\pm) \mathbf{W}_t}{\|\mathbf{T}_f(p_a^\pm) \mathbf{W}_t\|}. \quad (4.6)$$

Algoritmo 6: Clasificación *zero-shot* con CLIP

Entrada: Imagen objetivo I ; conjunto de clases $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$; parámetro de temperatura τ

Salida: Clase predicha $\hat{c}$

```

// Paso 1: Generación de los pesos del clasificador
1 para  $k = 1$  a  $K$  hacer
2    $p_k \leftarrow \text{«Una foto de »} + c_k$ ;
3    $\mathbf{T}_e^{(k)} \leftarrow \frac{\mathbf{T}_f(p_k) \mathbf{W}_t}{\|\mathbf{T}_f(p_k) \mathbf{W}_t\|}$ ;
4 fin
5 Almacenar en caché  $\{\mathbf{T}_e^{(k)}\}_{k=1}^K$ ;

// Paso 2: Extracción de la representación de la imagen
6  $\mathbf{I}_e \leftarrow \frac{\mathbf{I}_f \mathbf{W}_i}{\|\mathbf{I}_f \mathbf{W}_i\|}$ 

// Paso 3: Cálculo de similitudes y predicción
7 para  $k = 1$  a  $K$  hacer
8    $s_k \leftarrow \mathbf{I}_e \cdot \mathbf{T}_e^{(k)}$ 
9 fin
10  $\hat{k} \leftarrow \arg \max_k \text{softmax}\left(\frac{s_k}{\tau}\right)$ ;
11 devolver  $c_{\hat{k}}$ ;
```

Para una imagen I_m con embedding normalizado $\mathbf{I}_e^{(m)}$, la puntuación dicotómica se define como:

$$\delta_a^{(m)} = \cos\left(\mathbf{I}_e^{(m)}, \mathbf{T}_e^{a,+}\right) - \cos\left(\mathbf{I}_e^{(m)}, \mathbf{T}_e^{a,-}\right) = \mathbf{I}_e^{(m)} \cdot \mathbf{T}_e^{a,+} - \mathbf{I}_e^{(m)} \cdot \mathbf{T}_e^{a,-}. \quad (4.7)$$

Este valor final representa la **puntuación del atributo**. Al estar definido como una diferencia de similitudes, no debe interpretarse como una probabilidad. Valores positivos indican mayor alineamiento con el polo favorable del atributo, mientras que valores negativos indican mayor proximidad al polo desfavorable.

4.4.3. Ingeniería de *prompts* y ensamblaje

El diseño de las descripciones textuales es clave para la transferencia *zero-shot*. Los autores de CLIP señalan que la gran mayoría de los conjuntos de datos tradicionales consideran los nombres de las clases como una formalidad secundaria, anotando imágenes con identificadores numéricos simples. Esta falta de diseño textual plantea tres desafíos:

- **Polisemia.** Un problema común ocurre cuando el nombre de una clase tiene múltiples significados. Sin contexto, el codificador de texto de CLIP no puede diferenciar el sentido correcto de una palabra. Por ejemplo, la palabra *boxer* en inglés puede interpretarse como una raza de perro o un tipo de atleta. Proporcionar un contexto descriptivo

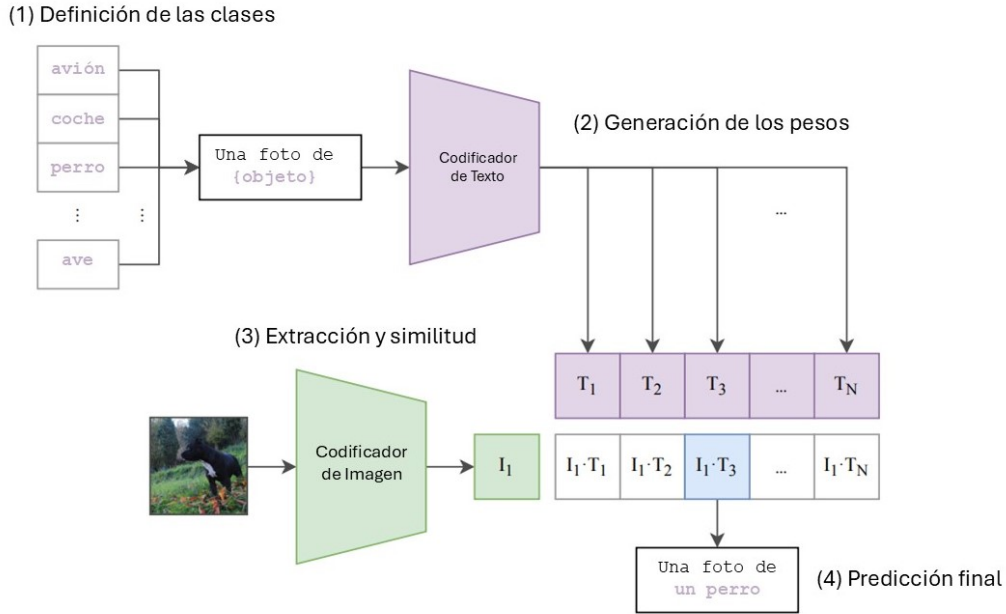


Figura 4.4 Uso de CLIP para predicción *zero-shot*. Primero se definen las clases de interés. A continuación se calculan los pesos del clasificador pasando las descripciones textuales por el codificador. La imagen objetivo se pasa por el codificador de imagen y se calcula su similitud con cada una de las clases. Finalmente, se escala mediante el parámetro de temperatura y el modelo predice la clase (imagen,texto) más probable a través de la función softmax. Imagen adaptada de [6].

elimina esta ambigüedad.

- **Brecha de distribución.** Es poco común que en el conjunto de datos de preentrenamiento de CLIP aparezcan textos compuestos por una sola palabra; lo habitual son oraciones completas que describen la imagen. Para cerrar esta brecha, los autores proponen el uso de plantillas como «Una foto de {clase}». Esta modificación básica mejora la precisión en ImageNet en un 1,3 % frente al uso de etiquetas aisladas [6].
- **Personalización por tarea.** El rendimiento mejora notablemente al personalizar el texto según el dominio. Por ejemplo, en tareas de clasificación de mascotas, añadir la coletilla «Una foto de {clase}, un tipo de mascota» ayuda a proporcionar el contexto necesario. Para el caso de fotos satelitales, especificar que se trata de «Una foto satelital de {clase}» resulta clave para el éxito del modelo.

Otra técnica relevante es el **ensamblaje de clasificadores**. Dado un conjunto de K *prompts* por clase, el embedding ensamblado se obtiene como:

$$\bar{T}_e^{(k)} = \frac{\sum_{j=1}^K \mathbf{T}_e^{(k,j)}}{\left\| \sum_{j=1}^K \mathbf{T}_e^{(k,j)} \right\|}. \quad (4.8)$$

En ImageNet, el ensamblaje de 80 plantillas aumentó la precisión en un 3,5 %, y combinado con la ingeniería de *prompts*, la mejora total alcanza casi 5 puntos porcentuales [6].

5. Metodología y diseño experimental

En este capítulo se aborda el objetivo central del trabajo: la construcción y validación de un **modelo de valoración inmobiliaria multimodal**. El término multimodal hace referencia al uso conjunto de fuentes de información de distinta naturaleza. En este caso, el modelo combina variables tabulares tradicionales, como superficie, número de estancias, planta o ubicación, con información visual extraída de las fotografías de los inmuebles mediante CLIP. El propósito es analizar si las imágenes contienen información predictiva adicional sobre atributos difíciles de cuantificar de forma directa, como el estado de conservación, la luminosidad, la calidad percibida de los acabados o las vistas.

La metodología experimental se plantea como un estudio progresivo de ablación. En primer lugar, se construyen modelos de referencia basados exclusivamente en variables tabulares. Posteriormente, se incorporan puntuaciones visuales obtenidas mediante CLIP y se evalúa si su inclusión mejora el rendimiento predictivo de forma estadísticamente significativa. Esta estrategia permite aislar la contribución de la información visual antes de introducir componentes más complejos, como la corrección no lineal de residuos analizada al final del capítulo.

En las secciones siguientes se describen la adquisición y limpieza de la base de datos, la estructura de las variables, la formulación de los modelos y la evaluación del papel de la información visual.

5.1. Exploración y preprocesamiento

5.1.1. Obtención de la base de datos

Para la validación empírica de este estudio, se requiere de un conjunto de datos que integre tanto variables estructuradas (características físicas del inmueble) como no estructuradas (fotografías). Dado que no existen repositorios públicos de libre acceso que combinen ambas modalidades, la base de datos fue construida *ad hoc*.

La base de datos fue obtenida mediante técnicas de *web scraping* ético aplicadas a la página web de [Fotocasa](#), delimitando el perímetro de búsqueda a la oferta de viviendas a la venta en el municipio de Gijón (Asturias). Esta base fue obtenida con fecha 19 de enero de 2026, con 21.700 imágenes correspondientes a 817 propiedades. La estructura final está planteada como una tabla en la que a cada vivienda se le asocia un identificador único (*id*), a través del cual se puede acceder a una carpeta donde están almacenadas sus imágenes.

5.1.2. Exploración y preprocesamiento de los datos

En esta sección nos centraremos en llevar a cabo el análisis exploratorio de la base de datos. La Tabla 5.1 contiene la descripción de las 24 columnas que la componen, así como el tipo de dato y el rango de cada una. El objetivo de la tarea de regresión es predecir la variable `price`, en torno a la cual se centra el estudio.

5.1.3. Distribución de la variable precio

Ohnishi et al. [33] analizó un millón de transacciones inmobiliarias y demostró que el cuerpo central de los precios de vivienda se ajusta a una distribución lognormal, excepto en períodos de burbuja inmobiliaria.

Como muestra la Figura 5.1, la distribución de precios está caracterizada por una gran cola a la derecha. Este fenómeno asimétrico es una propiedad habitual en variables de carácter económico sometidas a crecimientos multiplicativos, como ocurre con la distribución del salario y la riqueza de una población [34], o con los precios y retornos en los mercados financieros [35].

Se puede observar en la Figura 5.1 que la variable `price` sin transformar tiene una media de 318.112 € y una mediana de 240.000 €. Esta diferencia de 78.112 € es una consecuencia de la gran cola a la derecha de la distribución. La variable transformada sitúa su media en 12,46 y su mediana en 12,39. Al aplicar el logaritmo, ambas medidas de centralidad se han alineado y se ha aumentado la simetría de la distribución.

Este análisis justifica utilizar $\log(\text{price})$ como variable objetivo. La transformación logarítmica reduce la asimetría, comprime el efecto de los valores extremos y produce una escala más adecuada para modelos de regresión.

5.1.4. Detección de valores atípicos

La variable objetivo toma valores desde los 0 € hasta los 4.900.000 €. El primero de estos valores es, por supuesto, absurdo y se debe principalmente a precios de vivienda «a negociar». No obstante, estos valores añaden ruido y empeoran el rendimiento del modelo. Por otro lado, las viviendas de lujo extremo se rigen por una serie de normas distintas a las viviendas normales, y por tanto quedan fuera de los objetivos de este trabajo. En el análisis exploratorio de datos, este tipo de instancias se conocen como *outliers* pues se alejan significativamente del comportamiento general de la base de datos.

Tukey [36] propuso un método robusto para la detección de outliers, conocido como **método IQR**. Dada la distribución empírica de la variable precio en nuestro conjunto de datos $\mathcal{D}$, se definen el primer (Q_1) y tercer (Q_3) cuartil como los valores que acumulan, respectivamente,

Variable	Tipo de dato	Rango / Valores	Descripción
id	Numérica	817	Identificador único del anuncio.
address	Texto	-	Dirección del inmueble.
neighbourhood	Categórica	41 categorías	Barrio de ubicación.
district	Categórica	7 categorías	Distrito municipal.
latitud	Numérica cont.	[43,4530; 43,5579]	Coordenada de latitud.
longitud	Numérica cont.	[-5,7987; -5,5811]	Coordenada de longitud.
zipcode	Categórica	25 categorías	Código postal.
date	Fecha	[25/01/21, 19/01/26]	Fecha de publicación del anuncio.
scraped_date	Fecha	19/01/2026	Fecha de extracción de los datos.
price	Numérica cont.	[0, 4.900.000]	Precio de venta (€). Variable objetivo.
bathrooms	Numérica discr.	[0, 11]	Número de baños.
floor	Numérica discr.	[0, 21]	Planta del edificio.
air_conditioner	Booleana	{0, 1}	Disponibilidad de aire acondicionado.
heater	Booleana	{0, 1}	Disponibilidad de calefacción.
elevator	Booleana	{0, 1}	Edificio con ascensor.
swimming_pool	Booleana	{0, 1}	Disponibilidad de piscina.
rooms	Numérica discr.	[0, 12]	Número de habitaciones.
surface	Numérica cont.	[0, 7.000]	Superficie construida (m ²).
conservationState	Categórica	6 categorías	Estado de conservación.
garden	Booleana	{0, 1}	Disponibilidad de jardín.
terrace	Booleana	{0, 1}	Disponibilidad de terraza.
balcony	Booleana	{0, 1}	Disponibilidad de balcón.
parking	Booleana	{0, 1}	Plaza de garaje incluida.

Tabla 5.1 Descripción y rango de las variables del conjunto de datos tabular.

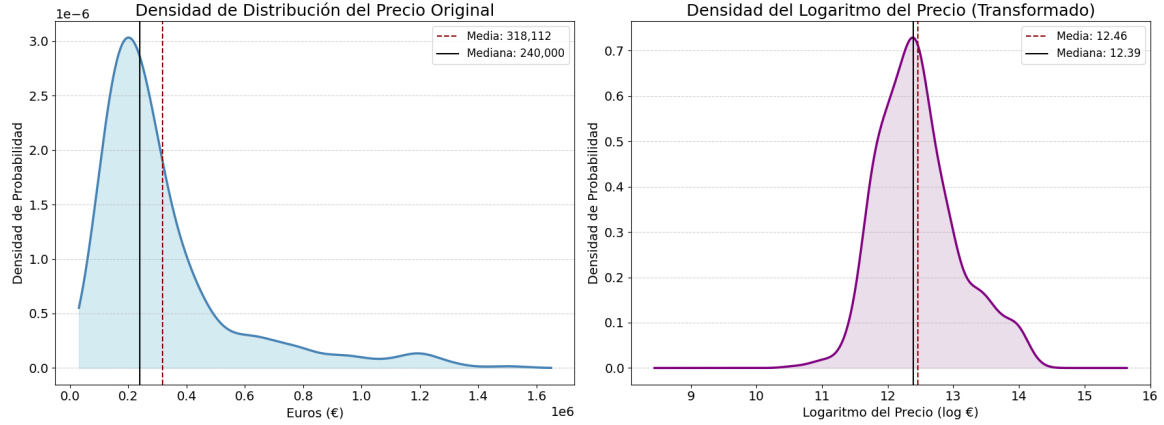


Figura 5.1 Distribución del precio de la vivienda en Gijón. Se observa una distribución marcada por una larga cola a la derecha (izda), habitual en variables con crecimiento multiplicativo. La distribución transformada (dcha) muestra un carácter mucho más simétrico y con medidas de centralidad agrupadas.

el 25 % y el 75 % de las observaciones. Sea $F_n(p)$ es la función de distribución acumulada empírica de los precios, los cuartiles se definen como:

$$Q_1 = \inf\{p \in \mathbb{R} : F_n(p) \geq 0,25\}, \quad Q_3 = \inf\{p \in \mathbb{R} : F_n(p) \geq 0,75\}. \quad (5.1)$$

A partir de estos estadísticos de orden, se calcula el **rango intercuartílico** (IQR , *Interquartile Range*), que captura la dispersión del 50 % central de la muestra:

$$IQR = Q_3 - Q_1. \quad (5.2)$$

Finalmente, se establecen los umbrales de aceptación inferior (L_{inf}) y superior (L_{sup}). Se considera que cualquier observación $x \in \mathcal{D}$ cuyo precio, denotado como $p(x)$, caiga fuera de este intervalo, constituye un valor atípico:

$$L_{inf} = Q_1 - 1,5 \cdot IQR, \quad L_{sup} = Q_3 + 1,5 \cdot IQR. \quad (5.3)$$

El cálculo con el método IQR genera los valores $L_{inf} = -127.500 \text{ €}$ y $L_{sup} = 652.500 \text{ €}$. No obstante, el límite inferior carece de sentido económico y, si se aplicase el método estrictamente, no eliminaría las viviendas «a negociar». Se define el conjunto final de valores atípicos como:

$$\mathcal{O}_{price} = \{x \in \mathcal{D} : p(x) \leq 0 \vee p(x) > L_{sup}\}. \quad (5.4)$$

Este conjunto contiene 87 elementos, que representan un 10,6 % de la muestra original.

5.1.5. Relación entre precio y superficie

La teoría de los modelos de precios hedónicos establece que el valor de una propiedad es la suma de los precios implícitos de sus características intrínsecas [37]. Dentro de estas ca-

racterísticas, la literatura señala sistemáticamente al tamaño del inmueble (**surface**) como el principal determinante de su valor de mercado [38]. Para comprender la dinámica entre ambas variables en nuestro caso de estudio, se ha optado por una representación visual que evalúa la concentración de la oferta así como la forma funcional de su relación (ver Figura 5.2).

El Panel A muestra la estimación de la densidad bidimensional, revelando dónde se concentra el núcleo de las ofertas inmobiliarias. Se aprecia un clúster principal de alta densidad (zonas rojas y granates) fuertemente concentrado en torno a viviendas de entre 70 y 90 m², con valoraciones que oscilan mayoritariamente entre los 120.000 € y los 230.000 €.

Por su parte, el Panel B expone el diagrama de dispersión superponiendo un ajuste de regresión local ponderada (LOWESS). Este modelo, propuesto por Cleveland [39], plantea una estimación obtenida mediante la minimización de la función de coste:

$$\sum_{i=1}^N w_i(x_0) (y_i - (\beta_0 + \beta_1 x_i))^2, \quad (5.5)$$

donde $w_i(x_0)$ es el peso asignado al dato i respecto al punto que queremos estimar x_0 . Estos pesos asignan más ponderación a los vecinos cercanos y se calculan habitualmente mediante la función tricubo:

$$w_i(x_0) = W\left(\frac{|x_i - x_0|}{d(x_0)}\right); \quad W(u) = \begin{cases} (1 - |u|^3)^3 & \text{para } |u| < 1 \\ 0 & \text{para } |u| \geq 1 \end{cases}, \quad (5.6)$$

siendo $d(x_0)$ la distancia al vecino más lejano dentro de la ventana local. La trayectoria resultante presenta una pendiente moderada en el segmento de inmuebles pequeños (hasta 60 m²), seguida de una aceleración o convexidad a partir de los 75 m². Esta evidencia sugiere que, para capturar fielmente el comportamiento de los precios, un algoritmo de regresión lineal debería incorporar términos polinómicos de orden superior para la variable superficie.

5.1.6. Análisis de la ubicación

Otro de los factores que más afectan al precio de una vivienda es su ubicación. En la Figura 5.3 se pueden observar las diferencias de precio entre distintas zonas. Como cabría esperar, zonas céntricas, Cimadevilla y aquellas cercanas a la playa presentan precios más elevados. También destacan por su alto precio zonas de carácter más lujoso como Somió y, en la periferia, como Cabueñes. No obstante, el encarecimiento en estas zonas se puede explicar por el mayor tamaño de las propiedades ofertadas, y no solo por la ubicación en sí. Por el contrario, zonas de carácter más humilde como El Llano o El Natahoyo presentan precios mucho más asequibles.

La distribución espacial de la oferta refleja el dinamismo del mercado en las zonas consoli-

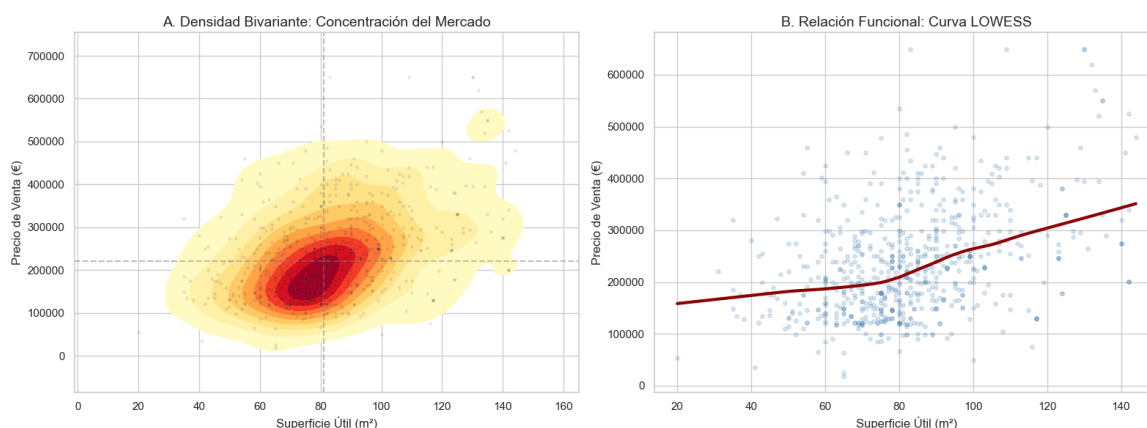


Figura 5.2 Análisis de la relación entre superficie útil y precio de venta. **(A)** Estimación de densidad bivalente: el núcleo de la oferta se concentra en viviendas de 70–90 m² con precios entre 120.000 y 230.000 € (zona roja-granate). Las líneas discontinuas marcan la mediana de cada variable. **(B)** Diagrama de dispersión con ajuste LOWESS: la curva muestra una pendiente moderada hasta los 60 m² y una aceleración a partir de los 75 m², evidencia de una relación no lineal que justifica la incorporación de términos polinómicos de orden superior para la variable superficie.

dadas: El Llano, Natahoyo, Centro y La Arena concentran la mayor parte de los anuncios. Las zonas periféricas como Cabueñes o Nuevo Rocés presentan una representación más escasa. Esto se explica tanto por su menor densidad de oferta como por el filtro IQR aplicado: $p > 652.500$ €. Las propiedades de mayor valor, que tienden a concentrarse en estas áreas, quedan excluidas de la muestra.

5.1.7. Análisis de características estructurales y equipamiento

Hasta ahora, el análisis exploratorio se ha centrado fundamentalmente en la superficie y la ubicación espacial de los inmuebles. No obstante, las variables tabulares correspondientes a características estructurales y de equipamiento (como el número de habitaciones, baños o la disponibilidad de ascensor) ejercen también un efecto determinante sobre la formación del precio final.

La Figura 5.4 presenta los diagramas de caja de las variables discretas y dicotómicas del modelo. El ascensor, la plaza de garaje y el balcón elevan claramente la mediana del precio. La piscina, en cambio, apenas la modifica, algo esperable en una región donde el clima limita su uso buena parte del año.

Por último, las variables de capacidad muestran una relación no lineal con el precio. En el número de baños, el salto más pronunciado se produce en la transición de dos a tres, mientras que en el número de habitaciones la diferencia más notable aparece al pasar de tres a cuatro estancias. Ambos patrones sugieren que superar el estándar de vivienda familiar se asocia a incrementos de precio mayores que los escalones anteriores.

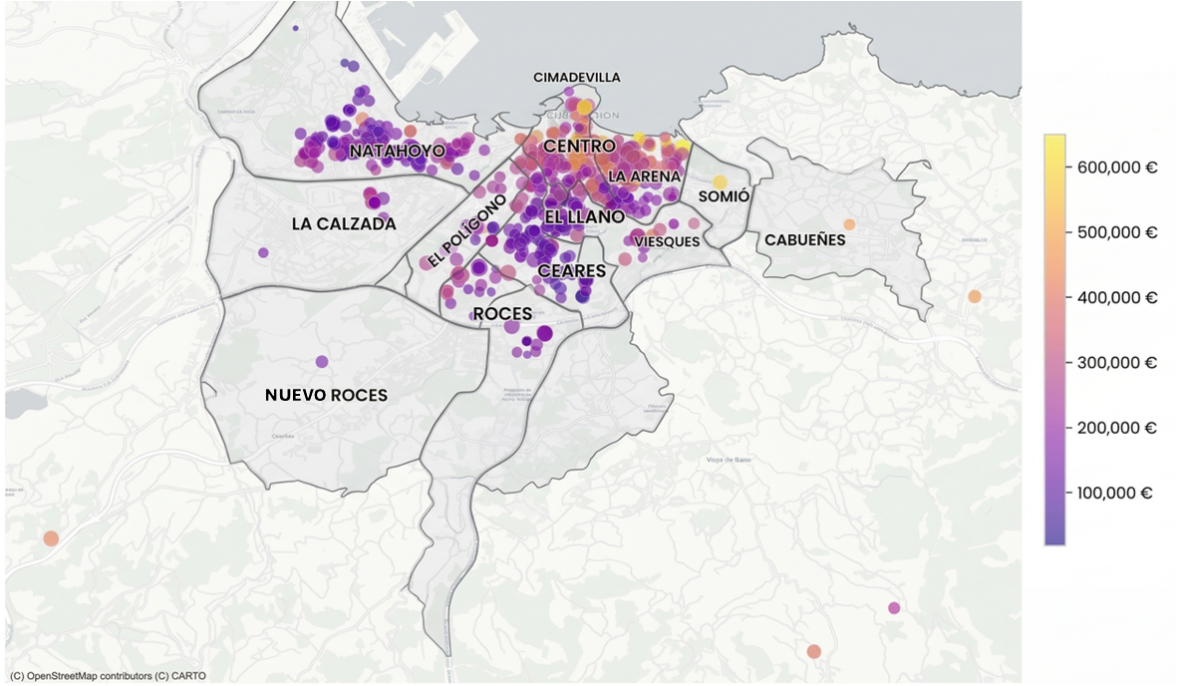


Figura 5.3 Distribución de precios en Gijón por zonas obtenida a través de nuestra base de datos. Las demarcaciones de los barrios son aproximaciones hechas en base a los registros municipales. Se puede observar que zonas como Centro, Somió o Cimadevilla tienen precios superiores a El Llano o La Calzada, barrios conocidos por su carácter obrero y precios más reducidos.

5.1.8. Normalización de variables

Tras la fase de exploración y filtrado de los datos, la última fase del preprocesamiento consiste en la estandarización de las escalas para variables numéricas y continuas. Como se deduce de la Tabla 5.1, variables como `surface` o `latitud` operan en rangos dispares.

En modelos de ML, particularmente en regresión lineal, la disparidad de escalas puede provocar inestabilidad numérica y sesgar la importancia de los coeficientes hacia las variables con mayor magnitud absoluta, dificultando la interpretabilidad. [40].

Para solventar esta problemática, se aplicó una variante del escalado mínimo-máximo a todas las variables numéricas, proyectando sus valores a un intervalo acotado continuo de $[-1, 1]$. Dada una observación x perteneciente a una variable X , su valor normalizado se calcula como:

$$x_{norm} = 2 \cdot \left(\frac{x - \min(X)}{\max(X) - \min(X)} \right) - 1. \quad (5.7)$$

Esta transformación lineal conserva el orden relativo de las observaciones y sitúa todas las variables numéricas en una escala común con valores tanto negativos como positivos, lo que resulta coherente con las puntuaciones visuales derivadas de CLIP.

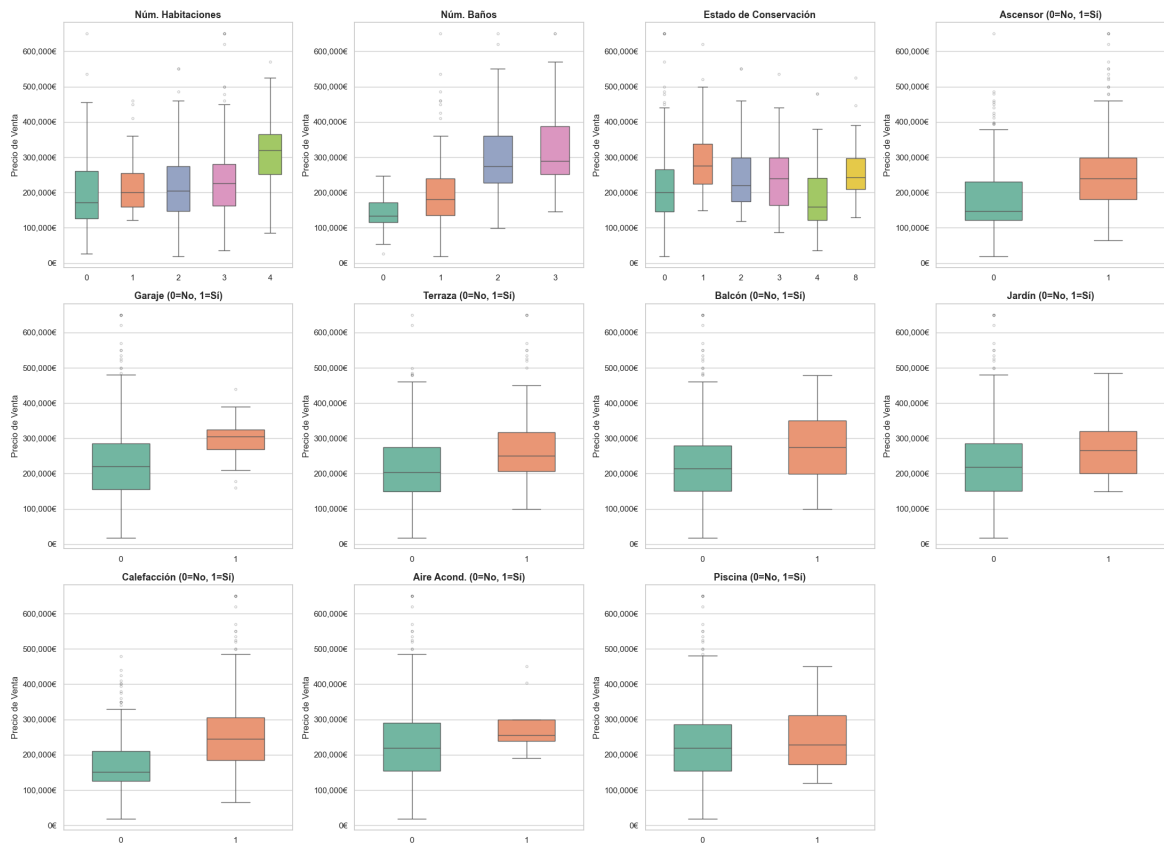


Figura 5.4 Impacto de las características estructurales y de equipamiento en el precio de venta. Diagramas de caja del precio según variables discretas (número de baños y habitaciones) y dicotómicas (ascensor, garaje, balcón, piscina). El ascensor, el garaje y el balcón elevan claramente la mediana; la piscina apenas la modifica.

5.1.9. Codificación de variables categóricas

De manera análoga al tratamiento de las magnitudes continuas, las variables de naturaleza cualitativa se transforman antes de ser alimentadas al modelo. Los algoritmos de regresión operan exclusivamente en espacios vectoriales numéricos, lo que imposibilita el procesamiento directo de atributos en formato texto, como el distrito municipal (`district`), el barrio (`neighbourhood`) o el estado de conservación (`conservationState`).

Para integrar esta información sin imponer un orden numérico artificial que el modelo pueda interpretar erróneamente como una jerarquía ordinal (es decir, asumir que el distrito 2 es el doble que el distrito 1), se aplicó la técnica de *One-Hot Encoding*. Este método transforma una característica nominal con k categorías exclusivas en k nuevas variables dicotómicas.

Dada una variable categórica C que toma valores en un conjunto discreto de categorías $\{c_1, c_2, \dots, c_k\}$, la transformación genera un vector de características $\mathbf{d} = [d_1, d_2, \dots, d_k]$

donde cada componente se define como:

$$d_i = \begin{cases} 1 & \text{si } C = c_i \\ 0 & \text{en caso contrario} \end{cases}. \quad (5.8)$$

Es fundamental destacar que introducir las k variables dicotómicas de manera simultánea genera un problema de multicolinealidad, provocando que la matriz utilizada para obtener la solución por mínimos cuadrados de la regresión sea no invertible. Este problema se conoce como **trampa de las variables ficticias** [41]. Para evitarlo, la codificación omite sistemáticamente una de las categorías, generando un espacio de dimensión $k - 1$. La categoría excluida asume el rol de nivel de referencia base y los coeficientes estimados por el modelo para el resto de categorías se interpretan directamente como el diferencial de precio esperado respecto a dicho nivel de referencia.

5.2. Metodología propuesta

En esta sección discutiremos la metodología propuesta para la creación, entrenamiento y evaluación del modelo. Como se describió en el Capítulo 4.4, CLIP se utiliza para generar una puntuación en el intervalo $[-2, 2]$ para una serie de variables visuales relevantes para un modelo de valoración inmobiliaria, como luminosidad, el estado de mantenimiento o la calidad de los acabados. La arquitectura de CLIP utilizada es la ViT-B/32 con pesos preentrenados `laion2b_s34b_b79k`.

Debido a que esta parte de *scoring* es pesada computacionalmente, se realizó aprovechando el servicio de [Google Colab](#) que pone a disposición de los usuarios unidades de procesamiento TPU v5e-1, capaces de acelerar enormemente este tipo de tareas.

5.2.1. Integración de características

Selección de dimensiones visuales

Como ya se detalló en la Sección 4.4.3, la capacidad del modelo CLIP para cuantificar atributos cualitativos depende del diseño de las instrucciones textuales. Para llevar a cabo la extracción de las características estéticas de las fotografías, se ha elaborado una rúbrica de evaluación fundamentada en ejes semánticos polares.

Cada dimensión visual de interés se representa como un espectro continuo acotado por un polo positivo, asociado a atributos de alto valor inmobiliario, y un polo negativo, relacionado con condiciones de deterioro o bajo valor. Para reducir la sensibilidad lingüística del modelo, cada polo se especifica mediante un conjunto de tres *prompts* sinónimos o complementarios, todos ellos formulados con la plantilla contextual «A real estate photo of...» para cerrar la brecha de distribución descrita en la Sección 4.4.3.

En una primera fase exploratoria, se diseñaron 14 dimensiones agrupadas en cuatro categorías: condición, espacio y luz, acabados y exterior. Tras el análisis de correlaciones entre dimensiones, se detectó una alta colinealidad entre atributos semánticamente próximos, por ejemplo, entre mantenimiento, actualización y habitabilidad, la cual comprometía la estabilidad de los coeficientes Ridge. Por ello, estas dimensiones se consolidaron en **6 macrodimensiones** que concentran la señal visual relevante y minimizan la redundancia. La Tabla 5.2 recoge la rúbrica final utilizada en el modelo. En el Apéndice A se recoge la versión completa de los *prompts* exactos utilizados.

Categoría	Dimensión Visual	Espectro Evaluado (Positivo ↔ Negativo)
Espacio y Luz	Luminosidad	Soleado ↔ Lúgubre
Acabados Base	Materiales	Parquet/pintura lisa ↔ Terrazo/gotelé
	Cocina	Alta gama ↔ Básica u obsoleta
	Baños	Reformado y moderno ↔ Viejo o deteriorado
Condición	Estado general	Impecable y habitable ↔ En ruinas
Exterior	Vistas	Panorámicas ↔ Bloqueadas

Tabla 5.2 Las 6 macro-dimensiones visuales evaluadas por el modelo CLIP, representadas como un espectro continuo entre sus anclas semánticas polares.

El modelo de puntuación CLIP se aplica sobre cada una de las viviendas de la base de datos, añadiendo 6 nuevas variables a la base original. Para cada casa, se asocia una única puntuación global a cada atributo, como se observa en la Figura 5.5. Este valor es la media de las puntuaciones obtenidas para todas las imágenes de la casa. Las casas sin imágenes reciben una puntuación neutra de 0 y se etiquetan en una nueva variable binaria **NoPhotos**. Esto resulta de vital importancia porque la ausencia de fotos representa un indicador negativo para una vivienda. Si simplemente se dejase el valor neutro 0, el modelo sería incapaz de distinguir esta alerta de una casa con simplemente una puntuación neutra en todos sus atributos.

5.2.2. Formulación del modelo

A partir de lo expuesto en las secciones previas, la predicción del valor inmobiliario se formaliza mediante un estudio de ablación estructurado en cinco modelos de regresión log-lineal múltiple. La ecuación general subyacente se define como:

$$\mathbf{y}^* = \beta_0 \mathbf{1}_N + \mathbf{X}_{\text{base}} \boldsymbol{\beta} + \mathbf{X}_{\text{fe}} \boldsymbol{\phi} + \mathbf{X}_{\text{clip}} \boldsymbol{\gamma} + \mathbf{X}_{\text{int}} \boldsymbol{\delta} + \varepsilon, \quad (5.9)$$

donde $\mathbf{y}^* = \log(\mathbf{y})$ es el logaritmo natural del precio y ε representa el término de error aleatorio. Para evaluar el aporte marginal de cada conjunto de datos, la matriz de diseño se construye de forma aditiva según el modelo evaluado:

- **M1: base.** Utiliza únicamente $\mathbf{X}_{\text{base}}$, que contiene las variables estructurales crudas, las variables categóricas de ubicación (ver Tabla 5.1) y la dicotómica indicadora de

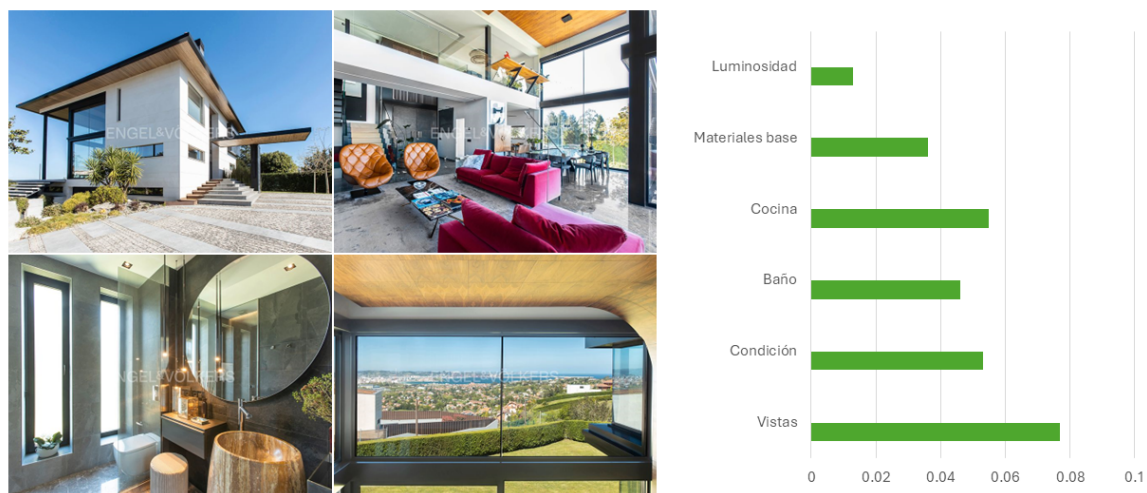


Figura 5.5 Puntuaciones visuales asignadas por CLIP a una vivienda concreta. El modelo asigna puntuaciones globales a una serie de 6 atributos predefinidos calculadas como la media de las puntuaciones sobre cada una de las imágenes. Las casas sin imágenes son etiquetadas con una variable de alerta.

ausencia de fotografías. Los vectores de parámetros ϕ , γ y δ se fuerzan a cero.

- **M2: base + *feature engineering* (FE).** Incorpora $\mathbf{X}_{fe}$, matriz que engloba transformaciones no lineales (como `surface2`), superficies de tendencia espacial (como `latitud × longitud`) y ratios estructurales (como `bathrooms / (rooms +1)` o `floor × elevator`).
- **M3: base + CLIP.** Excluye la ingeniería de características tabulares ($\phi = \mathbf{0}$) e introduce $\mathbf{X}_{clip}$, que contiene exclusivamente las puntuaciones de las dimensiones visuales extraídas mediante transferencia *zero-shot* (ver Tabla 5.2).
- **M4: base + FE + CLIP.** Modelo aditivo completo que evalúa la complementariedad entre el preprocesamiento matemático tabular $\mathbf{X}_{fe}$ y la extracción visual $\mathbf{X}_{clip}$.
- **M5: híbrido multiplicativo.** Supera la limitación aditiva introduciendo $\mathbf{X}_{int}$. Esta introduce interacciones entre métricas físicas y visuales (por ejemplo, `surface × macro_condition`).

Todas las variables generadas mediante ingeniería de características se encuentran recogidas en el Apéndice B.

5.2.3. Función objetivo

Por la alta dimensionalidad y la multicolinealidad que introducen las interacciones multiplicativas y las expansiones polinómicas, la estimación por **mínimos cuadrados ordinarios** puede resultar inestable [40]. Por ello, la estimación de los parámetros se realiza directamente mediante la minimización de una función de coste con regularización Ridge (ver Sección 3.5.1).

Con el fin de simplificar la notación para el modelo híbrido completo (M5), definimos una matriz de diseño global $\mathbf{Z}$ de dimensiones $N \times (1 + K)$, donde N es el tamaño de la muestra y K es el número total de predictores generados por la concatenación de todas las matrices de características:

$$\mathbf{Z} = [\mathbf{1}_N, \mathbf{X}_{\text{base}}, \mathbf{X}_{\text{fe}}, \mathbf{X}_{\text{clip}}, \mathbf{X}_{\text{int}}]. \quad (5.10)$$

De manera análoga, agrupamos el término independiente y todos los coeficientes en un único vector de parámetros $\boldsymbol{\theta} = [\beta_0, \boldsymbol{\beta}^\top, \boldsymbol{\phi}^\top, \boldsymbol{\gamma}^\top, \boldsymbol{\delta}^\top]^\top$. El estimador Ridge $\hat{\boldsymbol{\theta}}_R$ se obtiene resolviendo el siguiente problema de optimización:

$$\min_{\boldsymbol{\theta}} \left\{ \sum_{i=1}^N (y_i^* - \mathbf{z}_i^\top \boldsymbol{\theta})^2 + \alpha \sum_{j=1}^K \theta_j^2 \right\}, \quad (5.11)$$

donde $\alpha \geq 0$ es el hiperparámetro que controla la intensidad de la penalización L^2 aplicada a los coeficientes. Este problema de optimización posee una solución analítica cerrada, que se puede expresar como [40]:

$$\hat{\boldsymbol{\theta}}_R = (\mathbf{Z}^\top \mathbf{Z} + \alpha \mathbf{I}^*)^{-1} \mathbf{Z}^\top \mathbf{y}^*, \quad (5.12)$$

donde $\mathbf{I}^*$ es una matriz identidad de dimensiones $(1 + K) \times (1 + K)$ cuyo primer elemento diagonal es cero ($I_{11} = 0$), garantizando así la no regularización del intercepto [40].

5.3. Configuración experimental

Los valores concretos de los hiperparámetros utilizados, así como los detalles necesarios para la reproducibilidad del experimento, se encuentran detallados en el Apéndice C.

5.3.1. División del dataset

La base de datos original constaba de 817 observaciones. Al aplicar la eliminación del 10,6 % de valores atípicos, el tamaño muestral efectivo se reduce a 730 inmuebles. Esta decisión restringe el alcance del modelo al cuerpo principal del mercado inmobiliario y excluye operaciones atípicas, como precios simbólicos, anuncios «a negociar» o propiedades extremadamente singulares. En aprendizaje automático, este tamaño es reducido para una base de datos: una división simple sería inapropiada, ya que las métricas de desempeño dependerían en gran medida del subconjunto aleatorio elegido para el entrenamiento.

Para garantizar la robustez y reproducibilidad de los resultados, se implementa un esquema de validación cruzada de 10 iteraciones (ver Sección 2.2.3). El hiperparámetro de regularización α de la regresión Ridge se optimiza mediante una búsqueda en malla en cada iteración de la validación cruzada. En particular, se busca $\alpha \in [10^{-5}, 10^8]$ en donde el intervalo está generado

en escala logarítmica con 100 valores. Esto permite controlar el sobreajuste sin necesidad de segregar un tercer conjunto de validación externo.

5.3.2. Métricas de evaluación

Para cuantificar el rendimiento de los modelos y permitir su comparación, se emplean tres métricas estándar. Dado que la regresión se ajusta sobre la variable dependiente transformada logarítmicamente ($\mathbf{y}^*$), la bondad de ajuste inicial se evalúa en dicho espacio mediante el **coeficiente de determinación**:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i^* - \hat{y}_i^*)^2}{\sum_{i=1}^N (y_i^* - \bar{y}^*)^2}. \quad (5.13)$$

Esta métrica adimensional indica la proporción de la varianza del logaritmo del precio que es explicada por las variables independientes del modelo. Sin embargo, para evaluar el error de predicción en términos reales, es fundamental devolver las predicciones a la escala original.

Por la desigualdad de Jensen, $\exp(\mathbb{E}[\log(y)]) \leq \mathbb{E}[y]$, de modo que aplicar $\exp(\hat{y}^*)$ subestima sistemáticamente el valor real. Para corregir este sesgo, asumiendo que el término de error del modelo log-lineal se distribuye de forma normal $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, Wooldridge [42] demuestra que la media exacta se recupera incorporando la varianza residual muestral. Por tanto, las predicciones finales en euros ($\hat{y}_i$) se calculan como:

$$\hat{y}_i = \exp\left(\hat{y}_i^* + \frac{\hat{\sigma}^2}{2}\right). \quad (5.14)$$

Sobre estas predicciones corregidas $\hat{y}_i$ y los precios reales de mercado y_i , se calculan las métricas de error absoluto y error relativo:

Raíz del Error Cuadrático Medio (RMSE)

Penaliza de forma cuadrática las desviaciones más grandes, siendo especialmente sensible a los errores severos de tasación en propiedades de alto valor que no hayan sido filtradas como atípicas:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}. \quad (5.15)$$

Error Absoluto Medio (MAE)

Proporciona la magnitud promedio del error de tasación en euros, ofreciendo una interpretación directa y lineal sobre la precisión esperada del modelo. Al no elevar los residuos al cuadrado, es más robusto frente a predicciones extremas aisladas que el RMSE:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|. \quad (5.16)$$

Error Absoluto Medio Porcentual (MAPE)

Mientras que el MAE y el RMSE proporcionan la magnitud del error en la escala original (euros), resultan insuficientes para evaluar la precisión relativa teniendo en cuenta el rango de precio del inmueble. Un error absoluto de 40.000 € representa una desviación crítica en un inmueble de 50.000 € pero constituye un margen de precisión excelente para una propiedad del segmento *premium*.

El MAPE solventa esta limitación de escala al expresar la penalización como una fracción del precio real de mercado:

$$MAPE = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right|. \quad (5.17)$$

En el contexto de la tasación inmobiliaria, esta métrica resulta especialmente útil para detectar si el error se distribuye de forma homogénea a lo largo de la curva de precios o si se concentra en los extremos del mercado, algo que se analizará en detalle mediante la descomposición del error por deciles en el capítulo de resultados.

6. Resultados y discusión

En esta sección se evalúa el rendimiento predictivo de los cinco modelos de regresión Ridge.

6.1. Rendimiento predictivo

La Tabla 6.1 muestra los resultados obtenidos al entrenar cada uno de los modelos propuestos. Resulta notable observar que, tanto el modelo que incorpora variables visuales (M3), como el que incorpora ingeniería de características (M2), mejoran las métricas del *baseline*. Incorporar de forma conjunta (M4) ambas no mejora de forma significativa los resultados obtenidos por cualquiera de ellas por separado. Por último, el modelo que incluye más características (M5) presenta un coeficiente de determinación en entrenamiento considerablemente superior al obtenido en prueba. Esto es un síntoma de sobreajuste, y se puede deber a la excesiva complejidad introducida por las características de interacción frente al reducido tamaño muestral. La elevada desviación estándar del RMSE en este modelo (± 466 k€, casi el doble de la media) confirma esta hipótesis: en algunas particiones de la validación, el modelo comete errores extremadamente grandes al enfrentarse a inmuebles fuera del rango habitual de precios.

Modelo	R^2_{test}	R^2_{train}	MAE (k€)	RMSE (k€)	MAPE (%)
M1 (<i>baseline</i>)	$0,59 \pm 0,08$	$0,668 \pm 0,009$	58 ± 12	96 ± 59	$25,2 \pm 4,0$
M2 (base + FE)	$0,59 \pm 0,08$	$0,675 \pm 0,007$	57 ± 11	92 ± 50	$24,8 \pm 4,1$
M3 (base + CLIP)	$0,62 \pm 0,07$	$0,701 \pm 0,006$	56 ± 11	93 ± 55	$23,8 \pm 3,4$
M4 (base+FE+CLIP)	$0,62 \pm 0,07$	$0,706 \pm 0,007$	56 ± 12	95 ± 62	$23,7 \pm 3,5$
M5 (híbrido)	$0,58 \pm 0,19$	$0,727 \pm 0,007$	75 ± 62	243 ± 466	$27,0 \pm 14,0$

Tabla 6.1 Métricas de desempeño comparativas para los cinco modelos de regresión Ridge evaluados mediante validación cruzada. Los valores representan la media $\pm$ desviación estándar sobre los *folds*; en negrita se indica el mejor resultado por columna. La incorporación de las puntuaciones de CLIP (M3) produce la mejora más sustancial respecto al *baseline* (M1), elevando el R^2_{test} de 0,59 a 0,62 y reduciendo el MAPE en 1,4 puntos porcentuales. El modelo M4, que combina ingeniería de características y visión, iguala el rendimiento de M3 sin aportar una mejora diferencial. El modelo M5 evidencia un sobreajuste severo, con una brecha de 0,15 puntos entre R^2_{train} y R^2_{test} y una desviación estándar del RMSE (± 466 k€) que prácticamente duplica su media.

6.2. Contrastes de hipótesis

Para evaluar la significancia estadística de las diferencias en el rendimiento predictivo de los modelos se prescinde de tests paramétricos como el test t para datos pareados o el ANOVA.

En su lugar, se emplea el **Test de Wilcoxon de rangos con signo**. Este contraste no paramétrico evalúa si la mediana de las diferencias entre dos muestras pareadas es estadísticamente distinta de cero, basándose en el ordenamiento (rango) de la magnitud de dichas diferencias, lo que le confiere una alta robustez frente a valores atípicos.

Dados dos modelos de predicción A (modelo base) y B (modelo alternativo), para una muestra de N observaciones, se definen los vectores de error absoluto individual de cada modelo como $e_{A,i}$ y $e_{B,i}$, donde $i = 1, 2, \dots, N$. El test comienza con el cálculo de las diferencias para cada observación:

$$d_i = e_{A,i} - e_{B,i}, i = 1, \dots, N. \quad (6.1)$$

A continuación, se descartan las observaciones donde $d_i = 0$, reduciendo el tamaño muestral efectivo a $N_r \leq N$. Después, se ordenan los valores absolutos de las diferencias no nulas, $|d_i|$, de menor a mayor. A cada valor se le asigna un rango $R_i \in \{1, 2, \dots, N_r\}$. En caso de empates en $|d_i|$, se asigna el rango promedio correspondiente. Finalmente, se restituye el signo de la diferencia original a su rango correspondiente para calcular las sumas de rangos positivos (W^+) y negativos (W^-):

$$W^+ = \sum_{i=1}^{N_r} I(d_i > 0) R_i, \quad W^- = \sum_{i=1}^{N_r} I(d_i < 0) R_i. \quad (6.2)$$

En nuestro caso, el test se aplica sobre el vector completo de $N = 730$ predicciones, haciendo un contraste unilateral derecho. Es decir, se evalúa si el modelo alternativo reduce el error absoluto de forma consistente respecto al base:

- Hipótesis Nula (H_0): La mediana de las diferencias es menor o igual a cero. El modelo alternativo no reduce el error absoluto respecto al modelo base ($\text{Med}(d_i) \leq 0$).
- Hipótesis Alternativa (H_1): La mediana de las diferencias es estrictamente mayor que cero. El modelo alternativo reduce el error de forma consistente ($\text{Med}(d_i) > 0$).

El estadístico de contraste Z , se aproxima a una distribución normal para valores de N_r suficientemente grandes [43]:

$$Z = \frac{W^+ - \frac{N_r(N_r+1)}{4}}{\sqrt{\frac{N_r(N_r+1)(2N_r+1)}{24}}} \sim \mathcal{N}(0, 1). \quad (6.3)$$

Se fija un nivel de significancia $\alpha = 0,05$. Si el p -valor asociado al estadístico es inferior a α , se rechaza H_0 . La Tabla 6.2 recoge los resultados de este contraste. Se puede observar que el modelo M3 logra mejorar de forma significativa los resultados del modelo base, justificando el aumento de la complejidad. Se observa también que el modelo M5 es capaz de mejorar los resultados del M4, en contraste con lo que habíamos observado en la Tabla 6.1. El test de Wilcoxon trabaja sobre rangos, por lo que es robusto frente a *outliers*. M5 reduce el error mediano respecto a M4, pero los valores atípicos en su distribución de errores hacen que su

MAE y su RMSE sean peores.

Modelo Base	Modelo Alternativo	p -valor	Significativo ($\alpha = 0,05$)
M1	M2	0,0729	No
M1	M3	0,0205	Sí
M3	M4	0,1221	No
M4	M5	0,0009	Sí

Tabla 6.2 Resultados del test de Wilcoxon de rangos con signo ($N = 730$ predicciones). Se evalúa la reducción del error absoluto respecto al modelo base.

6.3. Diagnóstico y limitaciones del modelo M3

Por el principio metodológico de la Navaja de Ockham, el modelo en torno al cual se ha decidido centrar el estudio es el M3, debido a su sencillez y a su mejora frente al modelo base. En esta sección se realizará un análisis de los residuos del modelo, interpretaremos los coeficientes de la regresión y determinaremos las limitaciones a las que se enfrenta.

6.3.1. Análisis de residuos

La Figura 6.1 recoge el diagnóstico de residuos del modelo M3. En el gráfico cuantil-cuantil mostrado en la Figura 6.1a se observan desviaciones de los cuantiles teóricos en ambas colas de la distribución. Este hecho justifica analíticamente la elección de contrastes no paramétricos (Sección 6.2), al violarse el supuesto de normalidad.

Por su parte, la Figura 6.1b muestra sesgo sistemático dependiente del rango de precios: el modelo sobreestima sistemáticamente el valor en los deciles inferiores y lo infraestima en los superiores. Este fenómeno es síntoma de la «rigidez» estructural de la regresión Ridge. Al minimizar el error cuadrático global mediante una combinación estrictamente aditiva, el modelo se ajusta a la media del grueso de la muestra, perdiendo la capacidad de modelar las no linealidades presentes en los extremos del mercado.

Por último, la Figura 6.1c cuantifica este impacto, mostrando cómo la pérdida de precisión adopta una forma de «U». El modelo comete errores relativos masivos (MAPE superior al 50 %) en los inmuebles de menor valor, mientras que en el segmento *premium* el colapso se manifiesta a través de desviaciones absolutas (MAE) extremadamente elevadas.

6.3.2. Interpretabilidad de coeficientes y varianza explicada

Una de las ventajas de la regresión lineal es la interpretabilidad de sus parámetros. Debido a la normalización al intervalo $[-1, 1]$ aplicada en la Sección 5.1.8, la magnitud y el signo de

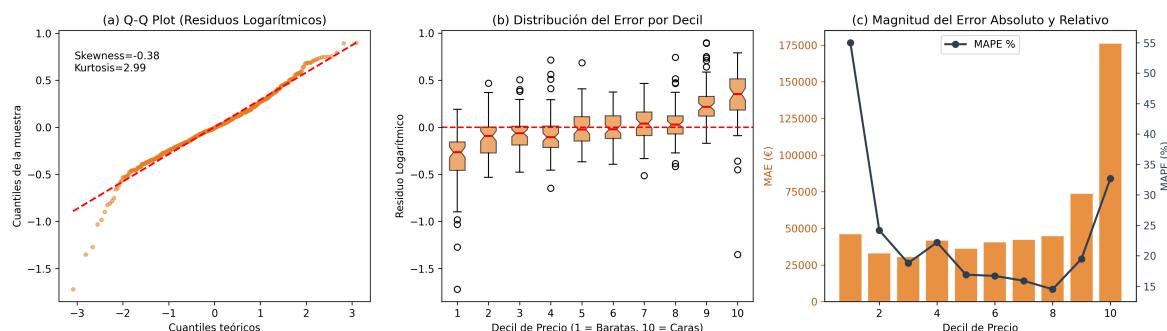


Figura 6.1 Análisis de errores del modelo M3. (a) Gráfico Q-Q de residuos logarítmicos frente a los cuantiles de una normal estándar; la diagonal representa el ajuste bajo el supuesto de normalidad. (b) Distribución del error por decil de precio real. (c) MAE (k€) y MAPE (%) por decil, con forma de «U».

los coeficientes estimados ($\hat{\beta}$) reflejan el impacto direccional y relativo que el modelo otorga a cada variable¹.

La Figura 6.2a muestra los coeficientes del modelo M3 ordenados por su magnitud absoluta. Como dictan los fundamentos del mercado inmobiliario, las variables de capacidad estructural (número de baños y superficie) dominan las primeras posiciones. No obstante, el rasgo más llamativo del ranking es el peso de la ubicación: de las 25 variables de mayor impacto, 16 corresponden a barrios o coordenadas geográficas (longitud y latitud), lo que refleja la fuerte segmentación espacial del mercado inmobiliario de Gijón². Los barrios con coeficiente negativo más pronunciado, como Veriña, Contrueces o Cenero, actúan como penalizadores de precio respecto a la categoría de referencia, mientras que la longitud captura ese gradiente Este-Oeste.

Para este estudio es especialmente relevante la aparición de dimensiones visuales de CLIP —como las «vistas» y «condición»— entre las quince variables de mayor peso del modelo. Esto confirma que la visión artificial captura y cuantifica atributos estéticos latentes que penalizan o revalorizan la vivienda

Por su parte, la Figura 6.2b desglosa la proporción de varianza explicada. El gráfico ilustra que la base tabular clásica absorbe la gran mayoría de la varianza predecible (58,8 %), confirmando la vigencia de los modelos de valoración tradicionales. El bloque visual aporta un incremento marginal neto del 3,5 %, pero la persistencia de una considerable fracción de varianza no explicada (~ 38 %) evidencia el límite de la aproximación lineal. Este remanente justifica la

¹Al emplear regularización Ridge, estos coeficientes se encuentran penalizados por lo que representan una contribución marginal condicionada al resto de variables y no un precio absoluto de mercado. En otras palabras, si el coeficiente de superficie es 0,5, no significa que cada metro cuadrado valga exactamente 0,5 unidades del precio. Significa que, dado que el modelo ya conoce el número de baños, la ubicación y el estado, el valor marginal que aporta específicamente el tamaño es ese.

²Como ya hemos discutido con anterioridad, existe una marcada polarización de precios entre la zona Oeste (La Calzada, El Natahoyo) y la zona Este (Somió, Cabueñes).

necesidad de evolucionar hacia arquitecturas no lineales capaces de capturar interacciones complejas.

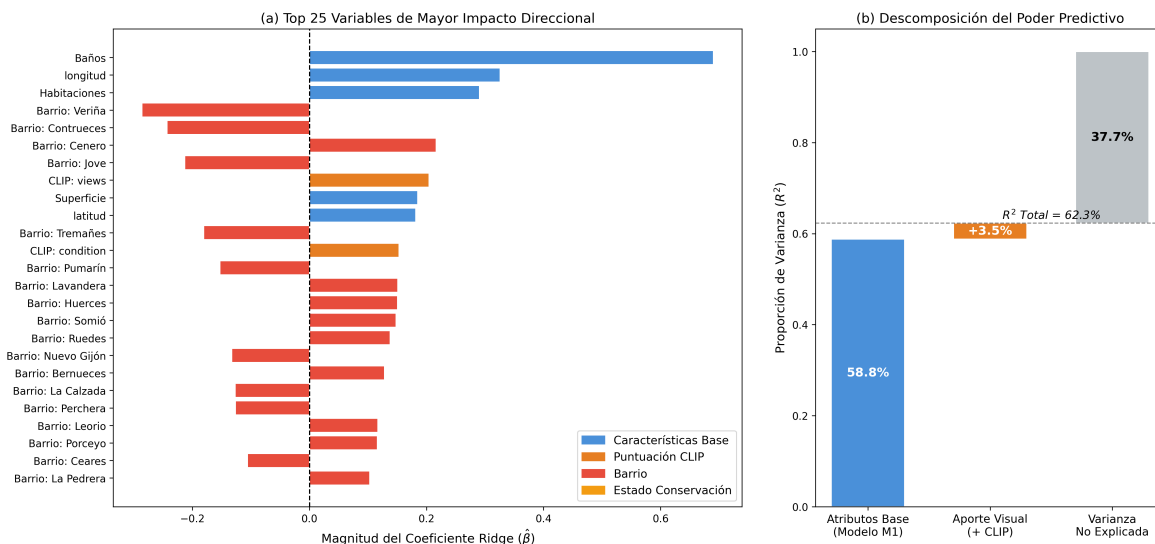


Figura 6.2 Interpretabilidad del modelo M3. (a) Top 25 variables ordenadas por magnitud absoluta del coeficiente Ridge ($\hat{\beta}$); el color distingue el tipo de variable. (b) Descomposición del R^2 : la base tabular aporta el 58,8 % de la varianza, el bloque visual añade 3,5 puntos adicionales y el 37,7 % restante queda sin explicar.

6.3.3. Análisis de inmuebles singulares

Como se ha advertido en secciones anteriores, una de las mayores limitaciones del modelo lineal es su vulnerabilidad ante anomalías del mercado. Estos inmuebles presentan condiciones excepcionales que rompen el supuesto de aditividad sobre el que se fundamenta la regresión Ridge.

Para aislar este fenómeno, se estudia la cola derecha de la distribución del error, aislando el percentil 95 superior de los errores relativos. Esto establece un umbral empírico de MAPE $> 57,8\%$. Bajo este criterio se identifican 35 viviendas críticas, de las cuales 34 representan sobreestimaciones severas y únicamente 1 resulta en infraestimación.

La Figura 6.3 expone las cuatro predicciones con mayor desviación de la muestra. Un análisis en detalle de estos fallos revela la existencia de factores de depreciación que el modelo es incapaz de advertir por sí solo, como inmuebles en subasta judicial, propiedades en estado de ruina o anuncios deliberadamente carentes de fotografías del interior.

El fracaso del algoritmo en estas instancias es una consecuencia de su proceso de optimización. Al minimizar la función de pérdida para el grueso de la muestra, la regresión ajusta sus coeficientes al comportamiento mayoritario del mercado. Ante una vivienda en ruinas de 190 m^2 , el modelo pondera sus grandes dimensiones sumando valor a la tasación de forma

independiente a su estado. Es incapaz de aplicar una penalización que anule el peso de la superficie cuando es inhabitable o cuando se encuentra en subasta.

Estos casos límite evidencian que incorporar matrices de datos tabulares y visuales no es suficiente si la arquitectura carece de no linealidad.



Figura 6.3 Ejemplos de anomalías donde el modelo M3 comete sus mayores sobreestimaciones. Para cada caso se muestra la fotografía del anuncio, el precio real, la predicción del modelo M3 y, en rojo, el factor de depreciación que el modelo es incapaz de capturar.

6.4. Construcción de un nuevo modelo

El análisis de errores del modelo M3 ha mostrado que la regresión lineal regularizada Ridge es muy eficaz para capturar la tendencia global del mercado inmobiliario. Sin embargo, carece de la flexibilidad necesaria para modelar efectos umbral, interacciones complejas o anomalías extremas en los extremos de la distribución de precios.

Para solventar esta limitación sin desechar la robustez del modelo lineal, se propone una arquitectura híbrida en cascada, que denominaremos modelo M6. Esta técnica divide el problema de predicción en dos etapas:

1. **Aproximación global** (lineal). El modelo M3 ajusta la variable objetivo $\mathbf{y}^*$ utilizando el espacio de características $\mathbf{Z}$, generando una predicción base $\hat{\mathbf{y}}_{M3}^*$.
2. **Corrección local** (no lineal). Se calculan los residuos de la predicción lineal en el conjunto de entrenamiento, $\mathbf{r} = \mathbf{y}^* - \hat{\mathbf{y}}_{M3}^*$. A continuación, se entrena un algoritmo no paramétrico diseñado para predecir este vector de errores.

La predicción final del modelo M6 se formula como la suma de ambos componentes:

$$\hat{y}_{M6,i}^* = \hat{y}_{M3,i}^* + H(\mathbf{x}_i), \quad (6.4)$$

donde $H(\mathbf{x}_i)$ es la función de aproximación aprendida por el segundo modelo para estimar el residuo de la observación i . Una ventaja de esta arquitectura es la preservación de la interpretabilidad. Un sistema de tasación debe poder descomponer el valor asignado a un inmueble: la regresión lineal captura la tendencia central del mercado mientras que el componente no lineal actúa como corrección residual. Así, se incrementa la capacidad predictiva sin sacrificar la transparencia de los coeficientes del modelo base.

6.4.1. Regresor de *gradient boosting*

Para la función de corrección $H(\mathbf{x})$, se emplea *XGBoost*. Este algoritmo de *gradient boosting* construye un ensamblaje aditivo y secuencial de árboles de decisión, destacando como uno de los modelos más eficaces para datos tabulares. La formulación matemática detallada de su proceso de optimización puede consultarse en el trabajo de Chen y Guestrin [44].

Al entrenarse directamente sobre los residuos del modelo M3, el ensamblaje presenta un elevado riesgo de sobreajuste por memorización del ruido de la muestra. Para evitarlo, los hiperparámetros se regularizan de forma estricta. La profundidad de los árboles se acota a $d = 4$, forzando predicciones simples y evitando una fragmentación excesiva de la muestra. La tasa de aprendizaje se fija en $\eta = 0,05$, penalizando la contribución de cada árbol a la corrección final y obligando al modelo a construir la estimación en base a muchos estimadores débiles. Finalmente, se introduce una penalización L^2 sobre los pesos de las hojas ($\lambda = 1$), análoga a la regularización Ridge del modelo base, que empuja las correcciones hacia cero, con mayor fuerza en las hojas con pocas observaciones.

El objetivo del Modelo M6 no es anular completamente el residuo de entrenamiento, sino corregir las grandes desviaciones observadas en los deciles extremos de la distribución de precios. La Tabla 6.3 recoge las métricas de rendimiento tras la agregación del ensamblaje de árboles, junto con el contraste de Wilcoxon evaluado frente a las predicciones base.

Los resultados demuestran una mejora global sustancial: el MAE se reduce en aproximadamente 8.000 € y el MAPE desciende por debajo del 19 %. El test de rangos con signo confirma, con un p -valor muy bajo ($p < 0,0001$), que esta reducción del error es consistente a lo largo de la muestra y estadísticamente significativa, apoyando la existencia de una mejora del modelo lineal a través de la arquitectura en cascada.

Modelo	R^2_{test}	R^2_{train}	MAE (k€)	RMSE (k€)	MAPE (%)
M3	$0,62 \pm 0,07$	$0,701 \pm 0,006$	56 ± 11	93 ± 55	$23,8 \pm 3,4$
M6	$0,71 \pm 0,07$	$0,892 \pm 0,003$	48 ± 11	89 ± 69	$18,9 \pm 3,1$

Estadístico de Wilcoxon: $W = 161.456$; $p = 3,58 \times 10^{-16}$ (significativo, $\alpha = 0,05$).

Tabla 6.3 Comparación de métricas entre el modelo de referencia M3 y la arquitectura en cascada M6. La corrección no lineal reduce el MAE en 8.000 € y el MAPE en 4,9 puntos porcentuales. La mejora es estadísticamente muy significativa según el test de Wilcoxon de rangos con signo.

La elevada desviación del RMSE (± 69 k€, un 77 % de la media) refleja la sensibilidad de esta métrica a los *outliers* persistentes en los deciles extremos, ya identificados en la diagnosis de M3. Por su parte, la brecha entre R^2_{train} (0,892) y R^2_{test} (0,71) evidencia un sobreajuste moderado, esperable al entrenar un ensamblaje de árboles sobre los residuos del modelo lineal.

El diagnóstico de residuos del modelo M6 revela sus límites. El diagrama de dispersión (Figura 6.4a) muestra un buen ajuste a lo largo de la diagonal para la mayor parte del mercado, aunque persisten algunos *outliers* extremos. El error por deciles (Figura 6.4b) mejora respecto a M3. El MAPE del primer decil baja del 55 % al 40 % y el MAE del décimo decil se reduce ligeramente. Aun así, la curva MAPE conserva su forma de «U», lo que indica que el modelo pierde precisión en los extremos del mercado: propiedades en muy mal estado o de lujo exclusivo. El precio de estos inmuebles depende de factores que la arquitectura actual no captura, ni con los datos tabulares ni con el análisis de imágenes de CLIP.

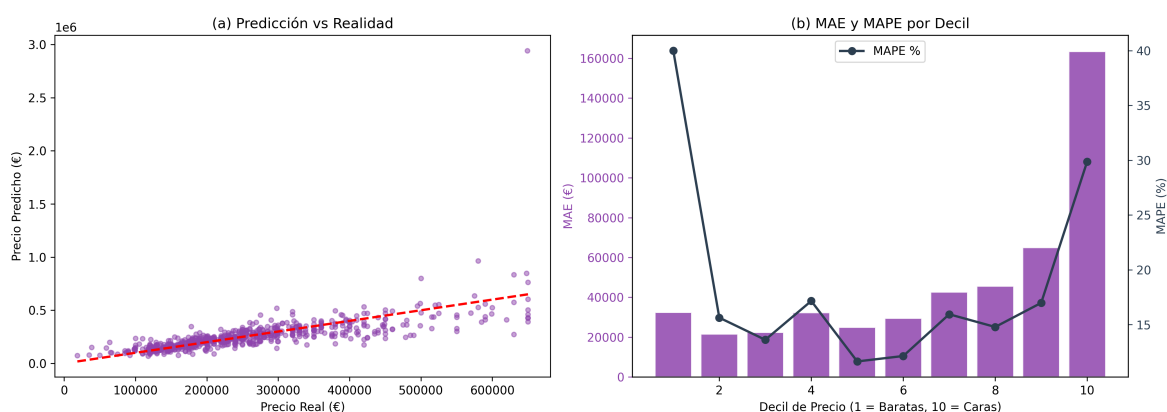


Figura 6.4 Evaluación de residuos del Modelo M6 en cascada. (a) Contraste entre el precio de mercado y la predicción final. La línea diagonal representa la predicción perfecta. (b) Distribución de las métricas de error absoluto y relativo por deciles de precio.

7. Conclusiones y líneas de trabajo futuro

Este capítulo resume los hallazgos principales del estudio y discute las líneas de trabajo futuro.

7.1. Conclusiones

- Las puntuaciones visuales generadas por CLIP son predictores estadísticamente significativos del precio de la vivienda. La incorporación de estas variables en el Modelo M3 mejora el coeficiente de determinación en test de 0,59 (R^2 del *baseline*) a 0,62, reduce el MAE de 58 k€ a 56 k€ y el MAPE del 25,2 % al 23,8 %. El test de Wilcoxon confirma la significancia de la mejora ($p = 0,0205$), mientras que la ingeniería de características clásica (M2) no logra diferenciarse del *baseline* ($p = 0,0729$).
- Las dimensiones «vistas» y «condición» extraídas por CLIP se sitúan entre las quince variables de mayor peso del modelo M3, indicando que los atributos estéticos subjetivos pueden codificarse en variables numéricas con capacidad explicativa.
- La descomposición de la varianza explicada por M3 sitúa la aportación de la base tabular clásica en el 58,8 %, frente al incremento marginal del bloque visual. Aproximadamente el 38 % de la varianza queda sin explicar, concentrándose en los deciles extremos: el MAPE supera el 50 % en el decil inferior y el MAE se dispara en el segmento *premium*. Este patrón en forma de «U» evidencia el límite de la aproximación lineal aditiva.
- La arquitectura híbrida M6 (Ridge + *XGBoost* sobre residuos) mejora todas las métricas frente al modelo lineal aislado: R^2 asciende de 0,62 a 0,71, el MAE se reduce un 14 % ($56 \rightarrow 48$ k€) y el MAPE desciende al 18,9 %. La significatividad estadística de esta mejora es muy alta ($p < 10^{-15}$). El diseño preserva la interpretabilidad del modelo lineal base mientras incorpora la capacidad de los árboles para capturar efectos umbral.
- La curva MAPE en «U» tras la corrección no lineal delimita el techo predictivo del sistema actual. El error remanente no es atribuible a la familia de modelos, sino a la ausencia de variables que describan circunstancias extratabulares del inmueble. El análisis cualitativo del percentil 95 de los errores identifica 35 viviendas críticas (34 sobreestimaciones, 1 infraestimación) determinadas por factores como subasta judicial, ruina estructural u ocultación deliberada de fotografías de interior, inaccesibles tanto para los datos tabulares como para el análisis visual.

7.2. Líneas de trabajo futuro

1. **Incorporación de NLP sobre descripciones textuales.** El análisis cualitativo de los *outliers* extremos muestra que la información necesaria para corregirlos aparece habitualmente en el texto del anuncio: menciones a «subasta», «para reformar», «ocupada» o «ruina». Aplicar técnicas de procesamiento de lenguaje natural sobre estos textos es la extensión con mayor expectativa de impacto sobre el decil inferior, donde se concentra el error remanente.
2. **Modelos multimodales de fusión temprana.** La arquitectura actual concatena representaciones extraídas de forma independiente. Una fusión temprana permitiría aprender interacciones cruzadas entre modalidades (por ejemplo, que una cocina obsoleta penaliza más en viviendas grandes que en pequeñas), un tipo de efecto que la aproximación aditiva no captura aunque incluya las variables por separado.
3. **Enriquecimiento con datos geoespaciales.** El peso elevado de la longitud geográfica y de los *dummies* de barrio en el modelo M3 sugiere que el componente espacial absorbe una fracción importante de la varianza. Variables como la calidad del espacio público, el ruido ambiental o accesibilidad al transporte podrían explicar parte del 38 % de varianza no explicada.
4. **Ampliación del conjunto de datos.** El tamaño muestral ($N = 730$ en test) explica parcialmente la varianza observada en los intervalos de confianza ($\pm 0,07$ en R^2 , ± 11 k€ en MAE) y el colapso del modelo M5 por sobreajuste ($R^2_{test} = 0,58 \pm 0,19$). Aumentar el tamaño muestral y validar la metodología en otros mercados permitiría evaluar su capacidad de generalización y desbloquear arquitecturas más complejas.
5. **Incorporación de variables macroeconómicas.** Los modelos desarrollados operan de forma seccional sobre un único momento temporal. Variables como el tipo de interés hipotecario, el índice de confianza del consumidor o la tasa de desempleo regional permitirían validar la metodología en contextos de alta volatilidad de mercado, donde el precio depende no solo de las características intrínsecas del inmueble sino también del entorno macroeconómico de la transacción.

A. Traducción de los prompts utilizados para la evaluación visual mediante CLIP

Este apéndice recoge los *prompts* utilizados para evaluar las dimensiones visuales resumidas en la Tabla 5.2. Cada dimensión se representa mediante dos conjuntos de anclas semánticas: *prompts* positivos, asociados a atributos visualmente favorables, y *prompts* negativos, asociados a atributos desfavorables. Las puntuaciones obtenidas mediante CLIP se interpretan como una medida relativa de proximidad semántica entre las imágenes del inmueble y cada extremo del espectro visual considerado.

Categoría	Dimensión	<i>Prompts</i> positivos	<i>Prompts</i> negativos
Espacio y luz	Luminosidad	<i>Interior con ventanas amplias y abundante luz natural; Interior luminoso y abierto bañado por el sol; Interior inundado de luz solar</i>	<i>Interior oscuro, sin ventanas y lúgubre; Interior con muy poca luz natural; Interior con iluminación artificial y espacio reducido</i>
Acabados base	Materiales	<i>Interior con parquet de madera nueva y paredes pintadas lisas; Interior con suelos modernos de primera calidad; Interior con materiales de construcción contemporáneos de alta calidad</i>	<i>Interior con suelo de terrazo antiguo y paredes de gotelé; Interior con linóleo barato u oxidado o baldosas dañadas; Interior con materiales de construcción obsoletos de los años setenta</i>
	Cocina	<i>Cocina moderna con armarios nuevos y electrodomésticos integrados; Cocina contemporánea recientemente reformada; Cocina con encimera de cuarzo o granito de alta gama</i>	<i>Cocina anticuada con armarios de madera viejos y electrodomésticos básicos; Cocina con azulejos de los ochenta y encimera de formica; Cocina deteriorada que necesita reforma completa</i>
	Baños	<i>Baño moderno reformado con ducha a ras de suelo; Baño con azulejos contemporáneos y griferías nuevas; Baño limpio y recientemente actualizado</i>	<i>Baño antiguo sin reformar con instalaciones originales; Baño con azulejos de colores anticuados y bañera vieja; Baño sucio que necesita grandes reparaciones</i>

Continúa en la página siguiente

Categoría	Dimensión	<i>Prompts</i> positivos	<i>Prompts</i> negativos
Condición	Estado general	<i>Interior completamente reformado y listo para entrar a vivir; Espacio de vida recién pintado e impecable; Interior en condición perfecta e intachable</i>	<i>Interior en estado de ruina que necesita vaciado completo; Interior abandonado con daños graves y pintura desconchada; Propiedad descuidada con deterioro estructural visible</i>
Exterior	Vistas	<i>Ventana con vistas exteriores despejadas y sin obstáculos; Vista panorámica a la calle o a la naturaleza; Vista amplia y luminosa desde la ventana</i>	<i>Ventana que da a un patio interior oscuro y cerrado; Vista bloqueada frente a una pared de ladrillo; Vista a un callejón estrecho y degradado</i>

Tabla A.1 *Prompts* utilizados para las 6 macro-dimensiones visuales evaluadas mediante CLIP. Los *prompts* originales están en inglés, pero han sido traducidos para facilitar la lectura. Cada entrada de la tabla representa el fragmento que completa la plantilla común. Todos los *prompts* llevan la plantilla «A real estate photo of [...]» delante.

B. Ingeniería de características

Se detallan a continuación las variables generadas a través del FE e introducidas en los modelos M2 y M5.

Variable	Formulación	Justificación teórica
<code>fe_surface_2</code>	surface^2	Modela la relación LOWESS creciente con el tamaño observada en el análisis exploratorio.
<code>fe_latitud_2</code>	latitud^2	Captura efectos espaciales no lineales Norte-Sur.
<code>fe_longitud_2</code>	longitud^2	Captura efectos espaciales no lineales Este-Oeste.
<code>fe_lat_lon</code>	$\text{latitud} \times \text{longitud}$	Modela interacciones espaciales diagonales.
<code>fe_ratio_bath_room</code>	$\frac{\text{bathrooms}}{\text{rooms} + 1}$	Aproxima el equipamiento relativo de la vivienda.
<code>fe_surface_room</code>	$\frac{\text{surface}}{\text{rooms} + 1}$	Mide la superficie media por estancia.
<code>fe_floor_elev</code>	$\text{floor} \times \text{elevator}$	Distingue el efecto de la altura según haya ascensor.
<code>fe_surface_terr</code>	$\text{surface} \times \text{terrace}$	Cuantifica el efecto del espacio exterior según el tamaño.
<code>fe_bathrooms_2</code>	bathrooms^2	Modela efectos no lineales del número de baños.

Tabla B.1 Variables tabulares sintéticas incorporadas en la matriz $\mathbf{X}_{fe}$. El factor +1 en los denominadores se introduce para prevenir singularidades.

Variable	Formulación	Justificación teórica
<code>int_surf_cond</code>	$\text{surface} \times \text{macro_condition}$	Modula el precio por metro cuadrado según el estado visual.
<code>int_surf_mat</code>	$\text{surface} \times \text{macro_materials}$	Escala el efecto de los acabados según el tamaño.
<code>int_floor_lum</code>	$\text{floor} \times \text{macro_luminosity}$	Cruza altura y luminosidad percibida.
<code>int_bath_bath</code>	$\text{bathrooms} \times \text{macro_bathroom}$	Ajusta el efecto del número de baños según su calidad visual.
<code>int_surfr_lum</code>	$\text{fe_surface_room} \times \text{macro_luminosity}$	Relaciona amplitud física y luminosidad percibida.
<code>int_surf_kitch</code>	$\text{surface} \times \text{macro_kitchen}$	Ajusta el efecto de la cocina según el tamaño.
<code>int_surf_view</code>	$\text{surface} \times \text{macro_views}$	Amplifica el efecto de las vistas en viviendas grandes.
<code>int_elev_cond</code>	$\text{elevator} \times \text{macro_condition}$	Cruza accesibilidad y estado general del inmueble.
<code>int_priv_mat</code>	$\text{fe_ratio_bath_room} \times \text{macro_materials}$	Relaciona equipamiento relativo y calidad de materiales.

Tabla B.2 Interacciones tabular-visuales incorporadas en la matriz $\mathbf{X}_{int}$.

C. Hiperparámetros y reproducibilidad

El código fuente completo de este trabajo está disponible de forma pública en el siguiente repositorio de GitHub: [multimodal-real-estate-valuation-model](https://github.com/anxohc/multimodal-real-estate-valuation-model).

Componente	Hiperparámetro	Valor utilizado
M1-M5: Estudio de ablación		
Reproducibilidad	Semilla aleatoria	<code>RANDOM_STATE = 42</code>
Validación cruzada	Número de particiones	<code>N_SPLITS = 10</code>
Validación cruzada	Estrategia de particionado	<code>KFold(n_splits=10, shuffle=True, random_state=42)</code>
Escalado	Método	<code>MinMaxScaler(feature_range=(-1, 1))</code>
Escalado	Ajuste	El escalador se ajusta solo con el conjunto de entrenamiento de cada fold
Ridge M1-M5	Estimador	<code>RidgeCV</code>
Ridge M1-M5	Malla de regularización	<code>$\alpha \in \text{np.logspace}(-5, 8, 100)$</code>
Ridge M1-M5	Selección de α	Selección automática dentro de la malla anterior en cada <i>fold</i>
Ridge M1-M5	Transformación inversa	$\hat{y} = \exp\left(\hat{y}_{\log} + \frac{\hat{\sigma}_{\text{res}}^2}{2}\right)$
M6: Regresión Ridge + XGBoost		
M6, fase Ridge	Estimador	<code>RidgeCV(alphas=np.logspace(-5, 8, 100))</code>
M6, fase no lineal	Estimador	<code>XGBRegressor</code>
M6, fase no lineal	<code>learning_rate</code>	0,05
M6, fase no lineal	<code>max_depth</code>	4
M6, fase no lineal	<code>min_child_weight</code>	15
M6, fase no lineal	<code>lambda</code>	1,0
M6, fase no lineal	<code>random_state</code>	42
M6, predicción final	Agregación	$\hat{y}_{\log}^{M6} = \hat{y}_{\log}^{\text{Ridge}} + \hat{r}_{\log}^{\text{GB}}$
M6, transformación inversa	Conversión a euros	$\hat{y} = \exp\left(\hat{y}_{\log}^{M6}\right)$

Tabla C.1 Hiperparámetros y configuración de reproducibilidad utilizados en la validación cruzada de los modelos.

Bibliografía

- [1] Idealista. *Evolución del precio de la vivienda en venta en España*. <https://www.idealista.com/sala-de-prensa/informes-precio-vivienda/>. [Accedido: 8 de marzo de 2026]. 2026.
- [2] idealista/news. *Se confirma que el precio medio de la vivienda marcó récord a cierre de 2025*. <https://www.idealista.com/news/inmobiliario/vivienda/2026/02/19/885211-se-confirma-que-el-precio-medio-de-la-vivienda-marco-record-a-cierre-de-2025>. Consultado el 17 de mayo de 2026. Feb. de 2026.
- [3] Ministerio de Vivienda y Agenda Urbana. *Estadística del valor tasado de la vivienda. Cuarto trimestre de 2025*. <https://apps.fomento.gob.es/BoletinOnline2/?nivel=2&orden=35000000>. Consultado el 17 de mayo de 2026. Gobierno de España, 2026.
- [4] Idealista. *Precio de la vivienda en Gijón*. <https://www.idealista.com/sala-de-prensa/informes-precio-vivienda/venta/asturias/asturias/gijon/>. [Accedido: 8 de marzo de 2026]. 2026.
- [5] Jerónimo Aznar Bellver y Ronny González Mora. *Valoración Inmobiliaria, Métodos y Aplicaciones*. Universidad Politécnica de Valencia, 2012.
- [6] Alec Radford et al. «Learning Transferable Visual Models From Natural Language Supervision». En: *arXiv preprint* (2021).
- [7] Pei Wang. «On Defining Artificial Intelligence». En: *Journal of Artificial General Intelligence* 10.2 (2019), págs. 1-37. DOI: [10.2478/jagi-2019-0002](https://doi.org/10.2478/jagi-2019-0002).
- [8] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep learning*. MIT press, 2016.
- [9] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. Cambridge, MA: MIT Press, 2012.
- [10] Simon J. D. Prince. *Understanding Deep Learning*. Available at <http://udlbook.com>. Cambridge, MA: MIT Press, 2023.
- [11] GeeksforGeeks. *Supervised vs Unsupervised vs Reinforcement Learning*. 2024. URL: <https://www.geeksforgeeks.org/machine-learning/supervised-vs-reinforcement-vs-unsupervised/> (visitado 30-10-2024).
- [12] Patrick Billingsley. *Probability and Measure*. 3rd. Wiley Series in Probability and Mathematical Statistics. John Wiley & Sons, 1995.
- [13] David H Wolpert. «The lack of a priori distinctions between learning algorithms». En: *Neural computation* 8.7 (1996), págs. 1341-1390.
- [14] W. S. McCulloch y W. Pitts. «A logical calculus of ideas immanent in nervous activity». En: *Bulletin of Mathematical Biophysics* 5 (1943), págs. 115-133.
- [15] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer, 2006. ISBN: 978-0387310732.

- [16] David E. Rumelhart, Geoffrey E. Hinton y Ronald J. Williams. «Learning representations by back-propagating errors». En: *Nature* 323.6088 (1986), págs. 533-536. DOI: [10.1038/323533a0](https://doi.org/10.1038/323533a0).
- [17] Kurt Hornik, Maxwell Stinchcombe y Halbert White. «Multilayer feedforward networks are universal approximators». En: *Neural networks* 2.5 (1989), págs. 359-366.
- [18] Walter Rudin. *Real and complex analysis*. 3rd. McGraw-Hill Education, 1987.
- [19] Amine ben khalifa e Hichem Frigui. «Multiple Instance Fuzzy Inference Neural Networks». En: *arXiv preprint arXiv:1610.04973* (2016). DOI: [10.48550/arXiv.1610.04973](https://doi.org/10.48550/arXiv.1610.04973).
- [20] Ashish Vaswani et al. «Attention is all you need». En: *Advances in neural information processing systems*. Vol. 30. 2017, págs. 5998-6008.
- [21] Daniel Jurafsky y James H. Martin. *Speech and Language Processing*. Draft. Disponible en: <https://web.stanford.edu/~jurafsky/slp3/>. 2026.
- [22] Alexey Dosovitskiy et al. «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale». En: *International Conference on Learning Representations*. 2021.
- [23] *Transformers from scratch*. <https://peterbloem.nl/blog/transformers>. Accedido: 11-03-2026.
- [24] Yann LeCun et al. «Gradient-based learning applied to document recognition». En: *Proceedings of the IEEE* 86.11 (1998), págs. 2278-2324.
- [25] Alex Krizhevsky, Ilya Sutskever y Geoffrey E. Hinton. «ImageNet Classification with Deep Convolutional Neural Networks». En: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 25. 2012, págs. 1097-1105.
- [26] Jia Deng et al. «ImageNet: A Large-Scale Hierarchical Image Database». En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2009, págs. 248-255. DOI: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- [27] Soumyajit Biswas, Anuvab Sen y Subhabrata Roy. «A review on early detection of Alzheimer's disease: employing deep learning, machine learning, and statistical methods». En: *CSI Transactions on ICT* 13.2 (2025), págs. 159-179. ISSN: 2277-9086. DOI: [10.1007/s40012-025-00416-w](https://doi.org/10.1007/s40012-025-00416-w). URL: <https://doi.org/10.1007/s40012-025-00416-w>.
- [28] Luca Deinger et al. «A comparative study between vision transformers and CNNs in digital pathology». En: (2022).
- [29] A. Kolesnikov et al. «Big transfer (BiT): General visual representation learning». En: *European Conference on Computer Vision (ECCV)*. 2020.
- [30] Richard Szeliski. *Computer Vision: Algorithms and Applications*. 2nd. Springer, 2022. URL: <https://szeliski.org/Book>.
- [31] Dhruv Mahajan et al. «Exploring the Limits of Weakly Supervised Pretraining». En: *CoRR* (2018). URL: <http://arxiv.org/abs/1805.00932>.

- [32] Qizhe Xie et al. «Self-training with Noisy Student improves ImageNet classification». En: *CoRR* abs/1911.04252 (2019). URL: <http://arxiv.org/abs/1911.04252>.
- [33] Takaaki Ohnishi et al. *On the evolution of the house price distribution*. Inf. téc. Presentado en la 58th World Statistics Congress of the International Statistical Institute (ISI). Working Paper, 2011.
- [34] Robert Gibrat. *Les inégalités économiques: Applications: aux inégalités des richesses, à la concentration des entreprises, aux populations des villes, aux statistiques des familles, etc., d'une loi nouvelle, la loi de l'effet proportionnel*. Librairie du Recueil Sirey, 1931.
- [35] Maury FM Osborne. «Brownian motion in the stock market». En: *Operations research* 7.2 (1959), págs. 145-173.
- [36] John W Tukey. *Exploratory data analysis*. Vol. 2. Reading, MA: Addison-Wesley, 1977.
- [37] Sherwin Rosen. «Hedonic prices and implicit markets: product differentiation in pure competition». En: *Journal of political economy* 82.1 (1974), págs. 34-55.
- [38] G Stacy Sirmans, David A Macpherson y Emily Norman Zietz. «The composition of hedonic pricing models». En: *Journal of Real Estate Literature* 13.1 (2005), págs. 1-44.
- [39] William S Cleveland. «Robust locally weighted regression and smoothing scatterplots». En: *Journal of the American Statistical Association* 74.368 (1979), págs. 829-836.
- [40] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. New York: Springer, 2009.
- [41] Damodar N Gujarati y Dawn C Porter. *Basic econometrics*. 5th. McGraw-Hill Education, 2009.
- [42] Jeffrey M Wooldridge. *Introductory econometrics: A modern approach*. 6th. Cengage learning, 2015.
- [43] William J Conover. *Practical nonparametric statistics*. 3rd. John Wiley & Sons, 1999.
- [44] Tianqi Chen y Carlos Guestrin. «XGBoost: A Scalable Tree Boosting System». En: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM. 2016, págs. 785-794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785). URL: <https://doi.org/10.1145/2939672.2939785>.